



HABITS;

심리 이론 기반의 자아 탐색과 **AI** 감정 분석을 통한
개인 맞춤형 습관 코칭 및 피드백 서비스

Team Leader	양설아	2276186
Team Member	Deng Yuanrong	2271003
Academic Advisor	심재형	
Submission Date	2026-03-26	2차 보고서

목차

- 1. 과제 요약**
 - 1.1. 프로젝트 배경 및 문제 정의
 - 1.2. 목표 및 해결 방안
 - 1.3. 기존 서비스와의 비교
 - 1.4. 제안 내용
 - 1.5. 기대 효과 및 의의
 - 1.6. 주요 기능 리스트
 - 1.7. 주요 기능의 엔진 및 설계
 - 1.8. 주요 기능의 구현
 - 1.9. 기타
- 2. 과제 설계**
 - 2.1. 요구사항 정의
 - 2.2. 상세 기능 명세
 - 2.3. 전체 시스템 구성
 - 2.4. 주요 엔진 및 기능 설계
 - 2.5. 개발 내용 및 현황
- 3. 팀 정보**
- 4. 향후 방향성**
- 5. 참고문헌**

1. 과제 요약

1.1. 문제 정의

배경 및 필요성

급변하는 사회적 환경 속에서 20대 청년 및 사회초년생들은 자신이 추구하는 가치와 일상의 행동을 일치시키는 데 점점 더 큰 어려움을 겪고 있다. 학업, 취업, 인간관계, 경제적 부담 등 다양한 스트레스 요인이 중첩되는 시기인 만큼, 단순한 의지력만으로 습관을 형성하고 유지하기에는 구조적인 한계가 존재한다. 이에 디지털 웰니스 솔루션에 대한 수요가 급증하고 있으며, Grand View Research¹에 따르면 전 세계 웰니스 앱 시장의 연 평균 성장률(CAGR)은 2025~2030년 사이 14.9%에 달할 것으로 전망된다.

그러나 이러한 시장 성장에도 불구하고, 기존 습관 형성 서비스에는 세 가지 근본적 문제가 존재한다.

문제점 1. 일률적 습관 추천, 개인 맞춤화 부족

기존 서비스 대다수는 개인의 성향, 성격적 특성, 생활 환경을 고려하지 않은 획일적인 습관을 추천한다. 사용자는 자신이 어떤 삶의 방식과 지향점에 가치를 느끼는지, 어떤 성격 특성을 강화하고 싶은지를 탐색할 기회 없이 범용적 루틴을 제안받으며, 이는 내재적 동기를 반영하지 못해 지속성을 담보하지 못한다는 문제를 야기한다.

문제점 2. 감정 상태와 습관 이행률간의 관련성 간과

스트레스 수준이 높아지면 습관 이행률이 급락하는 상관관계가 관찰되지만², 기존 서비스는 감정 상태와 무관하게 동일 강도의 습관 이행을 요구하며 사용자의 죄책감과 부담감을 가중시키는 경향이 있다. 이러한 압박의 결과는 사용자의 서비스 이탈로 이어진다.³

문제점 3. 초기 동기 상실과 낮은 지속률

단순히 습관 이행 여부를 기록하는 기능만으로는 초기 동기 상실이 발생하기 쉬우며, 사용자가 지속적으로 서비스를 이용하도록 유도하기에는 뚜렷한 한계가 있다. 구체적인 피드백 요소가 없는 어플리케이션을 사용자가 단순히 기록용으로 사용하다 금세 흥미를 잃고 이탈하는 현상이 빈번하게 보고된다.⁴

Target Customer와 Pain Points

HABITS 프로젝트가 정의한 타겟 커스트머는 자신이 어떤 사람이 되기를 원하는지 아직 잘 모르는 20대 젊은 층이다. 이들은 가치 있는 매일을 보내고 싶지만 어떤 삶의 방식이 자신에게 맞고 가치

¹ Grand View Research, "Wellness Apps Market Size, Share & Trends Analysis Report By Type, By Platform, By Device, By Subscription, And Segment Forecasts, 2025–2030", Grand View Research, 2024. 해당 보고서에 따르면 전 세계 웰니스 앱 시장 규모는 2024년 약 112.7억 달러(USD 11.27 billion)로 평가되었으며, 2025~2030년 연평균 성장률(CAGR) 14.9%로 성장하여 2030년까지 약 261.9억 달러에 달할 것으로 전망된다. <https://www.grandviewresearch.com/industry-analysis/wellness-apps-market-report> (접속일: 2026.03.27)

² Schwabe, L. & Wolf, O. T., "Stress Prompts Habit Behavior in Humans", 『Journal of Neuroscience』, 29(22), 2009, pp.7191–7198. 해당 연구에서는 스트레스에 노출된 피험자가 목표 지향적(goal-directed) 행동 대신 습관적(habitual) 반응에 의존하는 경향이 유의미하게 증가함을 실험적으로 입증하였다. 스트레스 집단은 통제 집단 대비 행동-결과 연합(action-outcome association) 인식률이 58%에서 28%로 급락하였다($p=0.017$). <https://www.jneurosci.org/content/29/22/7191> (접속일: 2026.03.27)

³ Business of Apps, "App Retention Rates 2026", 2026; Amra and Elma, "Top Mobile App Retention Statistics 2025", 2025. 해당 자료들에 따르면 모바일 앱의 평균 30일(Day 30) 유지율은 약 5~8% 수준으로, 설치 후 30일 이내에 전체 사용자의 90% 이상이 이탈하는 것으로 보고된다. 특히 생산성(Productivity) 앱의 경우 Day 1 유지율 17.1%에서 Day 30 유지율 4.1%로 급감한다. <https://www.businessofapps.com/data/app-retention-rates/> (접속일: 2026.03.27)

⁴ GetStream, "2026 Guide to App Retention: Benchmarks, Stats, and More", 2026. 해당 자료에 따르면 모바일 앱 사용자의 평균 Day 1 유지율은 28.29%에 불과하며, Day 7에는 17.86%, Day 30에는 7.88%로 급격히 감소한다. 미국 사용자 기준으로는 설치 후 30일 이내에 48%의 앱이 삭제되는 것으로 나타났다. <https://getstream.io/blog/app-retention-guide/> (접속일: 2026.03.27)

있는지 확신을 갖지 못한다. 즉, 본인 스스로도 잘 알지 못하는, 자신의 성향을 고려하지 않은 습관 서비스에서 본인이 어떤 것을 얻을 수 있는지 직관적으로 파악하기 어렵다고 느낀다. 또한 습관 앱을 사용한다 하더라도 그것을 통해 자신이 완성해나갈 결과물이 무엇인지 잘 모르겠다는 지점에서 혼란을 느낀다. 일정에 유동성이 발생하거나 컨디션에 변화가 생길 경우 습관 이행 자체가 부담이 되어 자주 미루게 되는 경우도 있으며, 구체적인 피드백이 없는 습관 서비스를 단순한 기록용으로 사용하다가 금세 사용을 중단하기도 한다. 이러한 **Pain Point**는 앞서 정의한 세 가지 문제점에 직접적으로 대응된다.

1.2. 목표 및 해결 방안

프로젝트의 목표

HABITS의 궁극적인 목표는 사용자가 의미 있는 행동 변화를 반복하여 자기 지향적 목표를 달성할 수 있도록 돕는 것이다. 이를 위해 본 서비스는 심리학 기반의 성향 진단, 사용자의 일상과 맥락을 반영한 맞춤형 루틴 추천, 그리고 감정 기반의 개인화 피드백을 결합한 통합형 코칭 서비스로 설계되었다. 사용자 여정은 진단에서 제안, 실행, 피드백으로 이어지는 순환 구조를 따르며, 이 네 단계가 유기적으로 작동하여 지속적인 행동 변화를 유도한다.

각 문제에 대한 해결 방안은 행동심리학 분야의 학술 연구에 기반하여 도출하였다. 1차 과제 리뷰 미팅에서 담당교수인 유광현 교수님으로부터 문제점과 해결 방안 사이에 논리적이고 객관적인 근거가 필요하다는 피드백을 받았으며, 이를 반영하여 관련 분야의 선행 연구를 조사하고 각 솔루션의 이론적 기반을 확보하였다.

다음은 프로젝트의 목표 달성을 위한 시스템에 대해 기술한 내용들이다.

자기결정이론(SDT) 기반 사용자 맞춤 습관 추천 시스템

첫 번째 문제인 일률적 습관 추천과 맞춤화 부족에 대해서는 Rochester 대학의 Ryan, R. M.과 Deci, E. L.이 발표한 「Self-determination theory and the facilitation of intrinsic motivation, social development, and well-being」(2000)에서 해결의 실마리를 얻었다. 해당 연구에 따르면, 개인이 외부에서 주어진 목표가 아니라 자신의 가치관과 일치하는 내재적 동기에 따라 목표를 맞춤 설정할 때 만족감과 유지력이 비약적으로 상승하며, 일률적인 시스템보다 개인의 선택권과 맞춤화가 부여된 환경에서 습관 형성 속도가 빠르다. 이에 기반하여 HABITS는 자체 성격 테스트를 통해 사용자의 현재 성향과 이상적으로 여기는 모습을 파악함으로써 내재적 동기를 진단하고, 사용자가 원하는 성격 특성을 해시태그로 선택하여 입력받아 해당 특성을 강화할 수 있는 습관을 추천하는 시스템을 설계하였다. 이는 일률적인 시스템이 아닌 개인의 선택권이 반영되고 맞춤화가 적용된 환경을 형성하는 것을 목표로 한다.

JITAI 모델 기반 감정-습관 통합 관리 시스템

두 번째 문제인 감정과 습관 이행률의 관련성 간과에 대해서는 Nahum-Shani, I. 등의 연구 「Just-in-Time Adaptive Interventions (JITAI)s in Mobile Health: Key Components and Design Principles for Ongoing Health Behavior Support」(2018)를 참조하였다. 해당 연구는 모두에게 동일한 알림을 보내는 기존 방식과 달리, 개인의 실시간 데이터에 기반해 '지금 이 순간 필요한' 맞춤형 선택지를 제공할 때 목표 달성률이 유의미하게 높다는 점을 밝히고 있다. 이는 행동심리학의 '맥락적 행동' 이론을 디지털 서비스에 적용한 사례이기도 하다. 이를 바탕으로 HABITS는 일기 쓰기를 통해 감정 데이터를 입력받아 사용자의 감정 상태를 파악하고, 습관 이행률 기록과 감정 데이터를 교차 분석하여 감정 상태와 습관 이행률 간의 관계를 진단한 뒤, 해당 관계의 양상에 따라 목표를 상향 혹은 하향 조정하거나 습관의 종류를 변경할 것을 제안하는 통합 관리 시스템을 구축하였다.

맞춤형 피드백 이론에 기반한 데이터 기반 AI 피드백 시스템

세 번째 문제인 초기 동기 상실과 낮은 서비스 지속률에 대해서는 네덜란드의 Dijkstra, A.와 De Vries, H.의 연구 「Personalized Persuasion: Tailoring Digital Health Interventions to User Characteristics」를 참조하였다. 해당 연구에 따르면, 금연이나 운동 습관 형성 시 일반적인 조언을 제공하는 것보다 사용자의 변화 단계에 맞춘 메시지를 전달했을 때 사기 저하가 훨씬 적고, 실제 행동 변화로 이어지는 비율이 2~3배 높게 나타난다. 이에 기반하여 HABITS는 주/월간 습관

이행률과 감정 상태 추이 데이터를 수집하고, 수집한 데이터를 교차 분석하여 사용자의 패턴을 파악한 뒤, 감정-이행률 상관관계, 변화 추세, 취약 시점 등 파악된 패턴을 기반으로 현재 상태에 맞는 구체적인 피드백을 제공한다. 여기서 구체적 피드백이란 달성률 변화의 원인 분석, 습관 강도 조정 제안, 그리고 개선 방향의 제시를 의미하며, 이러한 접근이 사기 저하를 예방하고 실제 행동 변화를 유도하는 데 효율적인 방법임은 선행 연구가 뒷받침하고 있다.

1.3. 기존 서비스와의 비교

HABITS의 차별점을 명확히 하기 위해, 현재 시장에서 널리 사용되는 습관 관리 서비스 2개를 선정하여 비교 분석을 수행하였다.

다른 서비스와의 비교

첫 번째 비교 대상은 루티너리(**Routinery**)이다. 루티너리는 2020년 국내에서 출시된 습관 및 루틴 관리 앱으로, 전 세계 500만 명 이상의 사용자를 확보하고 있으며 2025년 App Store '오늘의 앱' 선정, 2024년 포브스 헬스 'ADHD 베스트앱' 선정 등의 성과를 기록하였다. 루티너리의 핵심 강점은 타이머 기반의 순차적 루틴 실행에 있으며, 사용자가 사전에 설정한 루틴을 음성 안내와 자동 타이머로 안내하여 인지 부하를 줄이고 실행력을 높이는 데 특화되어 있다. 그러나 사용자의 심리적 성향을 사전 진단하는 기능이 없으며, 습관의 추천 자체보다는 사용자가 직접 등록한 루틴의 실행 보조에 초점이 맞춰져 있다. 감정 분석이나 AI 기반 피드백 기능은 제공하지 않는다.

두 번째 비교 대상은 **Fabulous**이다. **Fabulous**는 Duke University 행동경제학 연구소에서 인큐베이팅된 글로벌 셀프케어 코칭 앱으로, 전 세계 3,700만 명 이상의 사용자를 보유하고 있다. 행동과학에 기반한 단계적 습관 형성(Journey 시스템)과 오디오 코칭, 마음챙김 명상, 감사 저널 등 종합적인 웰빙 도구를 제공하며, 최근에는 AI를 활용한 적응형 프롬프트 기능도 일부 탑재하고 있다. 다만 **Fabulous** 역시 사용자 개인의 성격적 특성을 사전 진단하는 기능은 포함하지 않으며, 감정 분석을 위한 별도의 NLP 모델을 적용하지 않는다. 감사 저널이나 명상 등 감정 관련 도구를 제공하지만, 이를 습관 이행 데이터와 직접적으로 연계하여 분석하는 구조는 아니다.

HABITS의 핵심 차별화 포인트

위 두 서비스와의 비교를 통해 HABITS의 차별화 포인트는 크게 세 가지로 정리된다.

첫째, 성격 진단에 기반한 근본적 수준의 맞춤화이다. 루티너리와 **Fabulous** 모두 사용자의 심리적 성향을 사전 진단하는 기능을 제공하지 않는 반면, **HABITS**는 행동심리학에 기반한 자체 성격 유형 분류 체계를 설계하고, 사용자가 '현재의 나'와 '이상적인 나'의 간극을 인식한 뒤 그 간극을 좁힐 수 있는 습관을 추천받는 구조를 갖추고 있다. 이는 사용자가 직접 습관을 검색하거나 사전 설계된 코스를 선택하는 방식이 아니라, 내재적 동기에서 출발하여 습관이 제안된다는 점에서 본질적인 차이가 있다.

둘째, 감정과 습관 이행률의 직접적인 교차 분석이다. **Fabulous**는 감사 저널이나 명상 등 감정과 관련된 도구를 제공하지만, 해당 데이터를 습관 이행 실적과 연계하여 분석하는 기능은 갖추고 있지 않다. **HABITS**는 KoELECTRA 모델을 활용해 사용자의 일기 텍스트에서 감정을 정량적으로 추출하고, 이를 습관 이행률과 교차 분석하여 감정 상태에 따른 습관 강도 조정을 자동으로 제안한다. 이러한 감정-습관 연계 분석은 기존 서비스에서 찾아보기 어려운 고유한 기능이다.

셋째, 실제 행동 데이터와 감정 데이터를 종합 분석한 맞춤형 AI 코칭 피드백이다. 루티너리는 연속 성공일과 할일별 달성률 등 정량적 통계를 제공하고, **Fabulous**는 오디오 코칭 시리즈를 통해 동기를 부여하지만, 사용자 고유의 행동 패턴과 감정 추이를 분석하여 개인화된 텍스트 코칭 리포트를 생성하는 서비스는 현 시점에서 **HABITS**가 유일하다. **HABITS**는 비용 효율성을 확보하기 위해 Gemini API를 활용한 동적 메시지와 로컬 풀백 기반의 정적 메시지를 결합한 하이브리드 구조를 채택하였다.

1.4. 제안 내용

솔루션과 핵심 기능

앞서 정의한 세 가지 문제에 대하여 **HABITS**가 제안하는 솔루션과 그 핵심 기능을 요약하면 다음과 같다.

첫 번째 문제인 일률적 습관 추천과 맞춤화 부족에 대해서는 사용자 맞춤 습관 추천 시스템을 제안하며, 자체 성격 유형 테스트와 **Recombee** 추천 엔진을 핵심 기술로 활용한다. 두 번째 문제인 감정과 습관 이행률의 관련성 간과에 대해서는 감정-습관 통합 관리 시스템을 제안하며, **HuggingFace**의 **KoELECTRA** 감정 분석 모델과 이행률 교차 분석 로직을 핵심 기술로 활용한다. 세 번째 문제인 초기 동기 상실과 낮은 지속률에 대해서는 데이터 기반 **AI** 피드백 시스템을 제안하며, **Google Gemini Flash API**와 로컬 폴백을 결합한 하이브리드 구조를 핵심 기술로 활용한다.

이 세 가지 솔루션은 독립적으로 작동하는 것이 아니라, 진단(성격 테스트) → 제안(맞춤 습관 추천) → 실행(일별 이행 기록 및 감정 일기) → 피드백(**AI** 교차 분석 리포트)이라는 순환 구조 안에서 유기적으로 연결된다.

서비스에 대한 요구사항과 도입된 기능의 세부사항에 대해서는 ‘2.1. 요구사항 정의’와 ‘2.2. 상세 기능 명세’에서 후술한다.

담당 교수 리뷰 미팅 후 개선 사항

그로스 과목 담당 교수인 유광현 교수님과의 리뷰 미팅을 진행하며 여러 부분에 대한 피드백을 받았으며, 해당 내용들을 반영해 과제의 방향성과 세부 사항을 수정했다.

피드백 1: 과제 제목이 타겟 커스텀머와 **Pain Point**를 더 명확히 반영하도록 수정할 것을 제안

프로젝트의 기존 제목이었던 ‘자신의 약점을 보완하고 싶은 사람들을 위한, 맞춤 습관과 이행 관리, 그리고 사용자의 변화에 대한 피드백을 제공하는 서비스’를 ‘어떤 습관과 습관 형성 방식이 나에게 맞는 지 확신이 없는 젊은 층을 위한 성격 분석 기반 맞춤 습관 추천과 감정 분석-습관 이행률간 **AI** 교차 분석을 통해 개인화 피드백으로 원하는 성격 특성을 지속적으로 강화해나가는 습관 코칭 서비스’로 수정하여 목표 사용자와 문제점, 해결책을 더욱 명확히 보일 수 있도록 하였다.

피드백 2: 논리적·객관적 근거를 통하여 문제점과 해결책의 개연성을 확보할 것을 제안.

이를 반영하여 과제 제목을 현재의 형태로 수정 완료하였다. 둘째, 문제점과 해결 방안 사이에 논리적이고 객관적인 근거가 필요하다는 점을 지적하셨다. 이러한 피드백을 반영하여 자기결정이론(**SDT**), **JITAI**, **Dijkstra & De Vries** 연구 등 행동심리학 분야의 학술 논문을 조사하여 각 해결책의 개연성을 확보하였다.

피드백 3: 세부 기능에 대한 충분한 설명이 부족하므로 보충할 것을 제안

위와 같은 피드백을 반영하여 과제 수행 계획 발표 자료에서 세부 기능 페이지를 추가하는 방향으로 수정하였다.

1.5. 기대 효과 및 의의

HABITS가 성공적으로 구현될 경우 기대할 수 있는 효과는 다음과 같다. 우선, 내재적 동기에 기반한 맞춤 습관 추천과 감정 상태를 고려한 적응형 코칭을 통해 기존 습관 앱 대비 **30일 이상** 지속 사용률을 유의미하게 높이는 것을 1차 목표로 설정하고 있다. 또한 사용자가 자신의 감정 패턴과 습관 이행률 간의 상관관계를 구체적인 데이터와 리포트를 통해 확인함으로써, 서비스가 단순한 습관 기록 도구를 넘어 자기 이해의 도구로 기능할 수 있을 것으로 기대된다. 아울러, 이행률이 저조한 시기에 질책 대신 공감과 대안을 제시하는 피드백 구조를 통해 사용자가 '실패'를 '조정'으로 인식하게 만들어, 서비스 이탈률을 낮추고 장기적인 행동 변화를 유도하는 것을 목표로 한다.

본 프로젝트는 습관 형성 서비스에 행동심리학 이론(자기결정이론, **JITAI**)을 체계적으로 적용하고, **NLP** 기반 감정 분석과 생성형 **AI** 피드백을 결합하여 사용자의 정서적 맥락까지 고려하는 통합형 코칭 시스템을 제안한다는 점에서 의의가 있다. 특히 기존의 '행동 추적 중심' 습관 앱과 달리, '자기 이해 중심'의 접근을 채택함으로써 디지털 웰니스 시장에서 새로운 서비스 방향성을 제시한다. 또한 **Gemini API**와 로컬 폴백을 결합한 하이브리드 피드백 생성 구조는, 학생 프로젝트 수준에서 상용 **LLM API**의 비용과 속도 문제를 실무적으로 해결하는 접근으로서 엔지니어링적 의의 역시 갖는다.

1.6. 주요 기능 리스트

HABITS의 전체 기능을 서비스 흐름에 따라 리스트업하면 다음과 같다.

A. 온보딩 프로세스

A-1. 성격 유형 테스트: 자기결정론 기반 데이터 테이블로 사용자의 현재 성격과 사용자가 생각하는 이상적인 성격 유형 2가지 도출

A-2. 성격 특성 해시태그 선택: 각 유형에 할당된 4-5개의 해시태그 중 강화하고 싶은 특성을 복수 선택

A-3. 일상 루틴 입력: 통근, 식사, 수업/업무 시간 등 일상 패턴 입력

A-4. 맞춤 습관 추천: 해시태그와 루틴을 기반으로 한, ~~124개~~300개 습관 템플릿에서 적합도순 정렬 제시

A-5. 습관 일정 설정: 일상 패턴 반영 자동 제안과 사용자 직접 조절 가능

B. 습관 이행 관리

B-1. 일별 습관 체크리스트: 설정 일정 기반 자동 생성, 당일 이행 기록(재생 버튼을 누르면 해당 습관의 진행 시간 동안 링 타이머가 풀 스크린으로 제시되며, 타이머 완료 시 자동 완료로 기록. 수동으로 완료 이행 체크도 가능하도록 구현)

B-2. 주간/월간 이행을 요약: 전체 달성을 자동 집계

C. 감정 일기

C-1. 감정 슬라이더 입력: 기분 점수를 수치화하여 간편 기록

C-2. 감정 버튼 선택: 긍정적 키워드인 기쁨·평온·뿌듯·희망, 부정적 키워드인 슬픔·짜증·불안·피로 총 8개로 감정을 분류하여 키워드 버튼의 형식으로 제공함으로써 사용자가 빠르게 기록

C-3. 텍스트 일기 작성: 선택적으로 감정 일기를 텍스트로 기록

C-4. KoELECTRA 감정 분석: 텍스트에서 감정별 확률 분포 산출, DB 저장

D. 피드백 페이지

D-1. 달성도 탭: 주/월/연별 전체 달성률과 해시태그별 달성률 제공, 구간별 프로그레시브 메시지 제공

D-2. 감정 분석 탭: 감정 변화 추이 그래프, 감정-습관 이행을 상관관계 피드백 메시지 제공

D-3. 일기 탭: 기분 점수별 이모티콘, 일기 모아보기, 기간별 필터링 제공, 사용자가 감정 버튼으로 입력한 감정과 KoELECTRA 분석 감정을 색과 테두리를 통해 시각적으로 구분

E. AI 피드백 리포트

E-1. 주/월간 리포트 생성: Gemini Flash API와 로컬 풀백을 결합한 하이브리드 방식으로, 주간 리포트 자동 생성은 매주 일요일 22:00 월간 리포트 자동 생성은 매월 말일 22:00에 생성. 수동 리포트 생성은 사용자 트리거 시 항상 재생성되며, 해당 주/월 내에 이미 생성된 리포트가 있으나 다시 생성되는 경우 그 주의 리포트를 업데이트

E-2. 감정-습관 이행을 교차 분석: 상관관계, 변화 추세, 취약 시점 인사이트 제공. 이행을 구간을 5개로, 감정 **valence**를 4개로 나누어 **5×4**매핑 매트릭스를 형성해 기본 프레임으로 이용함. 변동성·추세는 거기에 추가적으로 분석하며, 그러한 내용을 바탕으로 종합 진단을 제시

E-3. 습관 강도 강도 조정 제안: 이행을 30% 미만일 시 빈도 축소/습관 변경, 90% 이상일 시 빈도 증가/습관 추가, 습관 졸업 안내

E-4. 맞춤 코칭 메시지: SDT 기반 페르소나의 공감적 피드백, 사용자 만족도 표기 피드백을 통해 리포트의 톤을 조정하고, 감정-습관 이행률간의 추세 관련 메시지 제공

F. 마이페이지

F-1. 일정 관리: 일정 루틴 수정, 습관 요일/시간 재조정

F-2. 해시태그 및 습관 변경: 성격 특성 해시태그 재선택, 습관 교체

F-3. 알림 설정: 습관 알림 및 피드백 리포트 알림

G. 사용자 인증

G-1. 회원가입: 아이디, 비밀번호를 입력받아 회원 가입

G-2. 로그인 및 자동 로그인: JWT 인증

G-3. 비밀번호 재설정: **Resend**를 통해 사용자에게 이메일을 발송함으로써 비밀번호 재설정

G-4. 회원 탈퇴: 데이터베이스에서의 **CASCADE** 삭제

H. 로컬-서버 동기화

H-1. **Local-First** 작성 및 수정: 오프라인 상태에서도 어플리케이션의 일부 기능을 이용할 수 있도록 설계

H-2. 양방향 동기화: **LLW** 방식을 통해 동일한 데이터에 대해 여러 번의 업데이트가 발생했을 때, 가장 최근의 타임스탬프를 가진 데이터를 최종값으로 채택하고 나머지는 버리는 방식을 사용

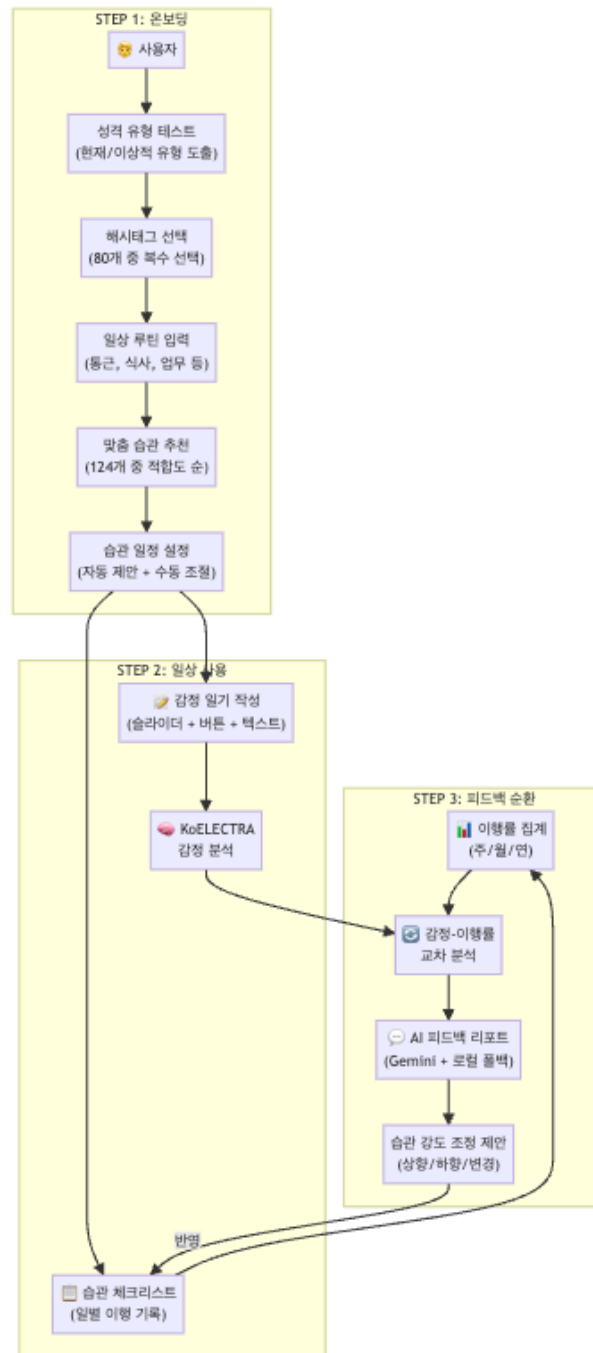
H-3. 다른 기기 로그인 시 자동 데이터 동기화: 서버에 데이터를 백업하여 동기화 진행

I. 만족도 평가 및 피드백 루프

I-1. 리포트 평가: 사용자로부터 피드백 리포트에 대한 만족도 평가를 받을 수 있는 버튼 제공

I-2. 만족도 추이 그래프 제공: 피드백 리포트 페이지에서 최근 90일간의 사용자 리포트 만족도 그래프를 제공

I-3. 사용자 피드백의 전반적 반영: 사용자 평가는 다음에 생성되는 리포트 톤과 **Gemini** 호출분기, **Recombee** 선호도 진단에 반영



<사용자 여정 기반 서비스 개념도>

1.7. 주요 기능 의 엔진 및 설계

1.7.1. 인증 엔진 (계정·세션 관리)

회원가입·로그인·토큰 갱신·비밀번호 재설정·계정 삭제를 담당. 단일 라우터(backend/app/routers/auth.py, 8개 엔드포인트)가 외부 인터페이스이고, 실제 보안 로직은 4개 하위 모듈로 분리되어 있다.

구성 모듈과 각 알고리즘

모듈	역할	알고리즘
auth/password.py	비밀번호 해싱·검증	passlib.CryptContext(schemes=["bcrypt"], bcrypt__rounds=12) — bcrypt cost 12. 1회 해시당 약 250ms (M1 기준) → 대량 brute-force 방어.
auth/jwt_handler.py	토큰 발급·검증	python-jose로 HS256 서명. _create_token(subject, token_type, expires_delta) 한 함수로 access/refresh/reset 토큰을 통합 생성하고 payload에 type 필드를 넣어 verify_token(token, expected_type)에서 타입 mismatch면 401. iat/exp 둘 다 UTC timezone-aware datetime.
auth/dependencies.py	FastAPI DI로 get_current_user 주입	OAuth2PasswordBearer로 헤더에서 토큰 추출 → verify_token(_, "access") → user_id로 DB 조회. is_active=False면 403.
auth/schemas.py	Pydantic I/O 스키마	EmailStr 검증, password: constr(min_length=8) 등 입력 단계에서 도메인 규칙 강제.

토큰 정책

- access: 15분 (ACCESS_TOKEN_EXPIRE_MINUTES=15)
- refresh: 7일 (REFRESH_TOKEN_EXPIRE_DAYS=7)
- password reset: 1시간 (PASSWORD_RESET_TOKEN_EXPIRE_HOURS=1)

DB 스키마 의존

- users: id, email(unique), password_hash, is_active, created_at
- 회원가입 시 같은 트랜잭션에서 Schedule(빈 레코드)·PersonalityResult(미응시 상태)는 만들지 않으며, 사용자가 온보딩에서 채우면 sync로 생성됨 (테이블 fan-out 최소화).

이메일 발송 (services/email.py)

- Resend API의 /emails POST 호출. 비밀번호 재설정 메일에 `https://app.habits/reset?token={reset_jwt}` 삽입.
- API 키 미설정 시 dict 형태 stub 반환 (개발 환경 graceful fallback).

1.7.2. 감정 분석 엔진

일기 텍스트 → 감정 분포 → 리포트 집계까지의 연결을 담당하는 엔진이다. 모델 추론은 클라이언트가 호출하고, 결과는 `sync`를 통해 서버 `EmotionAnalysis` 테이블로 들어와 리포트가 사용한다.

학습 데이터(AI Hub 감성 대화 말뭉치)의 원본 라벨은 60+개. 이 중 우리가 다루는 의미 영역에 해당하는 ~35개를 `LABEL_REMAP` 사전으로 8 클래스에 매핑한다.

```
LABEL_REMAP = {
  # joy
  "기쁨":"joy","행복":"joy","신남":"joy","즐거움":"joy","흥분":"joy",
  # calm
  "평온":"calm","안도":"calm","차분":"calm","편안":"calm",
  # proud
  "자신감":"proud","뿌듯":"proud","성취":"proud","자랑":"proud",
  # hope
  "희망":"hope","기대":"hope","설렘":"hope",
  # sadness
  "슬픔":"sadness","우울":"sadness","외로움":"sadness","공허":"sadness","비참":"sadness",
  # anger
  "분노":"anger","짜증":"anger","억울":"anger","분개":"anger","험오":"anger",
  # anxiety
  "불안":"anxiety","공포":"anxiety","걱정":"anxiety","초조":"anxiety","긴장":"anxiety",
  # fatigue
  "피로":"fatigue","지침":"fatigue","탈진":"fatigue","권태":"fatigue",
}
```

라벨 통합 정책의 근거는 다음과 같다.

- 8 클래스보다 더 세분화하면 클래스당 학습 샘플이 부족해져 일반화 성능 저하 (AI Hub 데이터 ~10K건 기준).
- 리포트의 `valence`(긍정/부정/혼합) 분류와 `narrative` 분기에 필요한 해상도가 8개로 충분.
- `LABEL_REMAP`에 없는 원본 라벨(예: "당황", "놀람")은 학습 시 건너뛴.

구성 모듈과 각 알고리즘

모듈	역할	알고리즘
frontend/src/services/emotionAnalysis.ts	HF Inference API 클라이언트	<code>analyzeEmotion(text, moodScore)</code> — fetch POST. 키 미설정·네트워크 오류·HTTP 비-2xx 모든 케이스에서 <code>{tags:[], scores:{}, sentiment: moodScore}</code> 반환 (graceful fallback, 기록 자체는 끊기지 않게).
monologg/koelectra-base-v3-discriminator (외부 모델)	한국어 감정 분류	8개 라벨(joy/calm/proud/hope/sadness/anger/anxiety/fatigue)

		로짓 → softmax → confidence 분포.
backend/app/feedback/keywords.py	모델 보조 신호	score_text_by_keywords(text) — 8 라벨 × 30+ 키워드 사전. 긴 키워드 우선 매칭(sorted(set, key=len, reverse=True))으로 "기쁘다"가 "기쁘"보다 먼저 매칭. merge_with_model_distribution(model_dist, text, model_weight=0.7) → 모델 70% + 키워드 30% 가중 평균. KoELECTRA confidence가 낮을 때 키워드가 보완 신호.
backend/app/feedback/constants.py	임계값, 라벨 정의	POSITIVE_EMOTIONS={joy, calm, proud, hope} / NEGATIVE_EMOTIONS={sadness, anger, anxiety, fatigue} / SINGLE_EMOTION_THRESHOLD=0.5 / COMBO_EMOTION_THRESHOLD=0.2 — 다른 모듈의 분기 기준.
backend/app/models/emotion.py (EmotionAnalysis)	분석 결과 영속화	(diary_id, main_emotion, confidence, distribution(JSON), analyzed_at). diary 1:1.
backend/app/sync/router.py::_upsert_emotion_analysis	클라이언트 결과 → 서버 저장	/sync/push의 diary item에 emotion_analysis 첨부 → 서버에서 diary upsert 직후 db.flush()로 id 확보 → EmotionAnalysis upsert.

데이터 흐름

[Diary 작성] (textContent) → analyzeEmotion(text, moodScore) # client → KoELECTRA via HF Inference API # 외부 → DiaryEntry.emotionAnalysis 저장 (zustand+AsyncStorage) → schedulePush(5s debounce) # client → POST /sync/push (diary + emotion_analysis) → _upsert_emotion_analysis() # backend → EmotionAnalysis 테이블 → 리포트 생성 시 aggregate_period() 가 distribution 평균 산출
--

1.7.3. AI 피드백 리포트 엔진

가장 모듈 의존성이 깊은 기능이다. 파이프라인은 8개 모듈을 순서대로 통과한다.

구성 모듈과 각 알고리즘

단계	모듈	알고리즘/입출력
1. 데이터 충분성 검사	analytics/conditions.py	is_weekly_data_sufficient() — 기간 내 활성 습관 수 ≥ 1 , 일기 수 ≥ 3 , expected_count ≥ 3 모두 충족 시 True. 미충족 시 리포트 생성 자체 skip.
2. 기간 집계	analytics/aggregation.py::aggregate_period(db, user_id, start, end)	_is_habit_due_on(habit, day) — frequency가 daily/weekdays/weekends/{type:"custom",days:[...]}에 따라 해당 일자 예정 여부 판정. expected/completed 카운팅 → completion_rate = completed/expected. mood: 일별 다일기 평균 → 시계열 mood_scores: list[float]. emotion: EmotionAnalysis.distribution을 누적 합산 후 합으로 정규화. 결과: PeriodAggregate 데이터클래스 (10개 필드).
3. 만족도 회수	feedback/satisfaction.py::get_recent_satisfaction(db, user_id, days=30)	FeedbackRating에서 최근 30일 평가 조회. _RATING_SCORE = {good:1, neutral:0, bad:-1} 매핑 후 평균. is_low = avg < LOW_SATISFACTION_THRESHOLD(-0.3). dominant_rating은 단순 최빈값.
4. 예외 패턴 감지	feedback/exceptions.py::detect_exceptions(rate, dist, mood_scores)	3개 sub-검출기 OR. (a) 역전: rate ≥ 0.80 AND neg ≥ 0.50 또는 rate ≤ 0.30 AND pos ≥ 0.70 . (b) 변동성: statistics.stdev(mood_scores) > 0.40 (데이터 ≥ 4 개일 때만). (c) 복합: confidence ≥ 0.20 인 감정 ≥ 3 개. 결과: ExceptionDetection(triggered, reasons[], detail{}).
5. 범주화	feedback/diagnosis.py::categorize_period_data()	4개 범주 + 추세. rate 5구간(0.20/0.40/0.60/0.80 경계), valence 4구간(pos-neg 차가 0.15 미만이고 둘 다 0.15 초과면 mixed), variance 3구간(0.20/0.40), mood 5구간, trend(이전 리포트 대비 ± 0.10). 결과: PeriodCategory.
6. 진단 매핑	feedback/diagnosis.py::build_diagnosis()	_DIAGNOSIS_MATRIX (5 $\times$ 4=20셀, 각 셀이 label·detail_template·keyword

		s) → 범주를 키로 즉시 결정적 lookup. 추세가 declining/improving이면 detail에 한 문장 추가, variance가 volatile이면 키워드 "기복" 추가.
7a. 진단 풍부화	services/gemini.py::enrich_diagnosis()	Gemini 2.0 Flash REST. category_summary(JSON)와 static_label/static_detail을 프롬프트에 동봉, 핵심 메시지는 유지하되 표현만 풍부화하라고 지시. temperature 0.7, topP 0.9, maxOutputTokens 250. 실패·미설정 시 None 반환 → 정적 detail 그대로.
7b. 품질 검사	feedback/quality.py::check_gemini_output(text)	정규식·substring 기반 자동 검사. 길이(20~250자), 금지 표현(해야 합니다, 반드시 등), 인과 단정(때문에, 로 인해), 이모지 카운트(r'[U0001F300-...]'). passed = score≥0.7 AND 금지표현 없음 AND 인과단정 없음. 통과 시 detail 교체, 실패 시 static_fallback.
8. 줄글 리포트 생성	feedback/narrative.py::build_narrative()	4문단 결정적 빌더. (1) 행동: expected/completed/rate. (2) 감정: top1·top2를 한국어 라벨로 자연어화. (3) 패턴: exc.reasons에 따라 분기 5종(역전·변동성·복합·긍정안정·부정저조). (4) 미달 습관: _find_lowest_completion_habit_info()가 반환한 가장 낮은 이행을 습관(<30%면 노출). (5) 만족도 톤: avg<-0.3이면 공감 한 줄, >0.3이면 감사 한 줄. recommendation은 bucket+valence+exc 조합으로 5종(add_habit/reduce_frequency/rest/null) 결정 후 target_habit_id 주입.
9. 본문 동적 생성 (조건부)	services/gemini.py::generate_feedback()	trigger: exc.triggered OR selection.needs_dynamic OR satisfaction.is_low. 프롬프트에 "톤 가이드" 섹션 동적 삽입 (만족도 hint). 품질 통과 시 full_text 교체, paragraphs는 단일 문단으로 대체.

1.7.4. 동기화 엔진

오프라인에서 쓰면 온라인에서 백업하고 이후에 합친다. 충돌은 LLW(Last-Write-Wins)로 해결한다.

구성 모듈과 각 알고리즘

모듈	담당	알고리즘
frontend/src/services/sync.ts	클라이언트 sync 매니저	pullSync() — since 쿼리에 last sync time 전달 → 서버 변경분만 회수 → habits/diaries/emotion_analyses 를 zustand 의 upsertHabits/upsertDiaries 로 머지. pushSync() — 전체 로컬 상태를 서버로 전송. schedulePush(5000ms) — debounce , 연속 변경을 1회 push 로 묶음.
backend/app/sync/router.py	서버 sync 엔드포인트	_to_utc(dt) 로 timezone-naive 를 UTC 로 정규화 후 _is_server_newer(server, client) 비교. server 가 더 최신이면 SyncConflict 반환· client 변경 폐기. push 처리 순서: habits → flush() → habit_logs (FK 참조 위해), 이후 diaries → personality → schedule .
backend/app/sync/schemas.py	sync I/O Pydantic	모든 sync target 에 공통: client_id : str(UUID), updated_at : datetime, deleted_at : Optional[datetime]. tombstone 패턴.
모든 sync target 모델 (Habit, Diary, PersonalityResult, Schedule, HabitLog)	DB 스키마	client_id UNIQUE per user , updated_at (LWW 키), deleted_at (soft delete). 외래키 cascade ondelete + passive_deletes=True .

LWW 알고리즘 의사 코드

<pre>def lww_apply(server_row, client_row): if server_row is None: insert(client_row); return None # 없으면 신규 if _is_server_newer(server.updated_at, client.updated_at): return SyncConflict(...) # 서버가 이김</pre>
--

```

overwrite(server_row, client_row)
return None

```

클라이언트가 이김

식별자 매칭

- Habit: (user_id, client_id) UNIQUE를 통해 같은 UUID로 동일 행 인식
- HabitLog: (habit_id, date) UNIQUE를 통해, 같은 날 중복 생성 방지. push에서 habit이 먼저 적용되어야 FK 참조 가능하며, db.flush()를 명시. (habit_id, date) UNIQUE를 통해 같은 날 중복 생성 방지. push에서 habit이 먼저 적용되어야 FK 참조 가능하며, db.flush()를 명시.

1.7.5. 추천 엔진

리포트의 “추천 습관” 라벨을 만드는 통계적 추천([narrative.py](#))과, 새 습관을 제안하는 협업 필터링(Recombee)이 분리되어 있다.

구성 모듈과 알고리즘

모듈	알고리즘
services/recombee.py	SDK lazy import. 키 미설정 시 모든 함수가 stub 반환. 도메인 매핑: view→AddDetailView(노출), bookmark→AddBookmark(채택), purchase→AddPurchase(이행). 명시 평가 send_rating(user_id, item_id, -1.0~+1.0) — good→+1.0/neutral→0/bad→-1.0.
routers/recommendations.py	/recommend/items → recommend_items_for_user(user_id, count) 호출. /interaction → 사용자 행동 이벤트 기록.
models/recommendation.py::RecommendationLog	추천 노출·수락·거부 로그. action enum 확장으로 rated_good/rated_bad 신호도 기록.
feedback/narrative.py	bucket(low/mid/high) × valence(긍정/부정/혼합) × exc.reasons 매핑으로 5종 결정. target_habit_id (lowest completion habit의 client_id) 동봉 → 프론트가 어떤 habit을 수정창에 prefill할지 결정.

1.7.6. JITAI (Just-In-Time Adaptive Intervention)

스케줄·이행 상태에 맞춰 푸시 알림 timing을 결정하는 엔진. V1 시점에서는 알림 인프라(FCM)가 미연결이지만 timing 결정 로직은 백엔드에 구현되어 있다.

구성 모듈

- routers/jitai.py: /api/v1/jitai/next-window?user_id → 다음 window 반환.
- 의존: models/schedule.py::Schedule(wake/bed/lunch/work/commute 시간), Habit.time_slot(morning/afternoon/evening/anytime).
- 알고리즘: time_slot을 user의 schedule 윈도우와 교집합으로, 비어있는 30분 슬롯 first-fit 선택. 이미 완료된 habit은 skip.

1.8. 주요 기능의 구현

1.8.1. AI 리포트 생성 흐름

Step 1. 스케줄러 발화

- `services/scheduler.py::start_scheduler()`가 앱 부팅 시 `BackgroundScheduler` 인스턴스를 1회 생성·start.
- 일요일 22:00 KST에 `_job_weekly()` 호출 → `today - timedelta(days=today.weekday())`로 이번 주 월요일 계산 → `generate_weekly_reports_for_all(db, this_monday)` 위임.

Step 2. 사용자별 루프

- `report_generator.py::generate_weekly_reports_for_all()`가 `User.is_active=True` 전체 조회 → 각 사용자에게 대해 `generate_weekly_report_for_user(db, u.id, this_monday)` 호출. 예외는 사용자 단위로 catch (한 사용자 실패가 전체 배치를 막지 않게).
- `generate_weekly_report_for_user(): _has_existing_report()` 중복 체크 → `_build_report_record()`.

Step 3. 본 빌더 `_build_report_record()`의 8단계 호출 체인

```
1. is_weekly_data_sufficient(db, user_id, week_start)      # analytics/conditions
   → 미충족이면 None 반환, 사용자 skip
2. agg = aggregate_period(db, user_id, week_start, week_end) # analytics/aggregation
3. exc = detect_exceptions(rate, dist, mood_scores)        # feedback/exceptions
4. satisfaction = get_recent_satisfaction(db, user_id, 30)  # feedback/satisfaction
5. previous = AIReport.query.filter(period_start<...).first() # 추세용
6. cat, diagnosis = build_diagnosis(...)                   # feedback/diagnosis
7. enriched = enrich_diagnosis(...)                       # services/gemini
   if quality.passed → diagnosis.detail = enriched
8. narrative = build_narrative(... satisfaction_avg=...)   # feedback/narrative
   if exc.triggered or selection.needs_dynamic or satisfaction.is_low:
       gemini_msg = generate_feedback(... satisfaction_hint=...)
       if check_gemini_output(gemini_msg).passed:
           full_text = gemini_msg
return AIReport(...)
```

Step 4. 영속화·열람 추적

- `db.add(report)`, 사용자 루프 끝에 `db.commit()`.
- `AIReport.read_at = NULL` 상태로 저장 시 미열람으로 간주.

Step 5. 클라이언트 노출

- `HomeScreen.tsx useFocusEffect`에서 `getUnreadCount()` → 헤더에 빨간 점과 숫자 배지를 표시.
- `ReportScreen.tsx`가 `listReports()` 호출해 카드 리스트에 NEW 배지 표시.
- 카드 탭 → `selected = report` 모달 오픈, 동시에 `markReportRead(id).then(load)` 호출하여 `AIReport.read_at = now()` 실행.
- 모달 본문은 `paragraphs[]` → `diagnosis_detail`, keywords 칩 → recommendation 카드 → 만족도 평가 버튼 순서로 배치.

1.8.2. 만족도 평가 피드백 루프

Step 1. 사용자 평가

- ReportScreen 모달 하단의 good/neutral/bad 버튼으로 services/feedback.ts::rateFeedback() 호출.
- 백엔드 routers/feedback.py::rate_feedback가 FeedbackRating(user_id, period_type, period_start, period_end, rating, source, comment) 저장.

Step 2. 다음 리포트 생성 시 영향

1. Gemini 호출 트리거: _build_report_record()의 use_gemini = exc.triggered or selection["needs_dynamic"] or satisfaction.is_low(만족도 < -0.3)이면 정적 메시지 반복이 원인일 수 있다고 판단해 Gemini 호출로 분석 시도.
2. Gemini 톤 가이드 주입: satisfaction.is_low면 hint = "사용자가 최근 N개 리포트 중 평균 X로 부정적 평가... 더 공감적이고 구체적인 어조로". dominant_rating == "good"이면 "긍정적 평가... 단조롭지 않게 새로운 관점 추가". _build_prompt()가 hint를 톤 가이드 섹션으로 동적 삽입.
3. 정적 narrative 톤 조정: build_narrative(... satisfaction_avg=...)가 < -0.3이면 공감 한 줄, > 0.3이면 감사 한 줄을 paragraphs에 추가.

Step 3. Recombee 신호

- 같은 엔드포인트에서 AIReport.recommendation.habit_id 조회하고, send_rating(user_id, str(habit_id), +1.0/0/-1.0) 결과를 파악하여 협업 필터링 학습에 신호를 줌.

1.8.3. 일기 작성을 통한 감정 분석 시 리포트 반영 흐름

Step 1. 일기 저장 (클라이언트)

- DiaryWriteScreen에서 textContent(감정 일기, 텍스트)와 moodScore 입력.
- useAppStore.addDiary({...})에서 zustand store 변경하고, persist 미들웨어가 AsyncStorage에 즉시 직렬화함으로써 일기를 저장함.

Step 2. 감정 분석 호출

- 같은 화면에서 analyzeEmotion(textContent, moodScore) 호출 → HF Inference API로 KoELECTRA 추론.
- 응답 파싱 진행 후 {mainEmotion, confidence, distribution, analyzedAt} → useAppStore.updateDiaryEmotion(diaryId, ...) 다시 AsyncStorage 직렬화.

Step 3. 백엔드 동기화

- 작성 직후 schedulePush(5000) 트리거 발생 시작 뒤 5초 후 pushSync() 호출하여 /api/v1/sync/push.
- 페이로드에 diaries[i].emotion_analysis 동봉하고 백엔드 상에서 _lww_apply_diary() → db.flush() → _upsert_emotion_analysis() → EmotionAnalysis의 과정을 거쳐 테이블에 (diary_id, main_emotion, confidence, distribution) 저장.

Step 4. 리포트 생성에 활용

- 다음 리포트 생성 시 `aggregate_period()`가 해당 기간 `EmotionAnalysis.distribution`을 누적·정규화 (`agg.emotion_distribution`)
- 이후 `categorize_period_data()`가 `valence(positive_dominant/negative_dominant/mixed)` 결정하고, `diagnosis matrix` 셀을 통해 라벨·detail·keywords 결정.

1.8.4. 다기기 동기화 흐름

Step 1. A기기 변경

- 사용자가 습관 빈도 수정을 진행하면 `updateHabitFrequency(id, freq)` 호출하여 업데이트하고, `updatedAt = new Date().toISOString()`을 통해 수정한 내용을 갱신.
- 서버 `_lww_apply_habit()`: 기존 행 발견 시 `_is_server_newer(server.updated_at, client.updated_at)` 을 통해 비교하고, 클라이언트가 더 최신이라면 `overwrite`함.

Step 2. B기기 fullSync

- B기기가 포그라운드 진입 시 `useFocusEffect`로 `pullSync()` 트리거 발생시킴.
- `lastSyncedAt` 이전 변경분만 회수하기 위해 `since=lastSyncedAt` 쿼리.
- 서버는 `Habit.updated_at > since`만 반환하고, 변경된 `habit` 1건이 응답에 포함.
- 클라이언트 `upsertHabits()`가 `id == client_id`로 매칭하여 같은 행에 `overwrite`함. `setLastSyncedAt(server_time)`로 다음 `since`를 갱신함.

Step 3. 충돌 시

- A·B가 동시에 같은 `habit` 변경 시 늦게 `push`된 쪽이 `server.updated_at`보다 `client.updated_at`가 작아짐. `_is_server_newer == True` 통해 서버가 `SyncConflict` 응답하면, 클라이언트는 `pullSync()`로 서버 최신값을 다시 받아서 병합함.

1.8.5. 401 만료 시 자동 갱신 흐름

Step 1. access 토큰 만료

- 임의 API 호출 시 401 응답을 반환함.
- `api.ts response interceptor`가 `catch`하여 확인함.

Step 2. race lock

- 동시에 여러 401이 발생할 수 있어 `let refreshPromise: Promise<boolean> | null = null` 단일 `promise`를 공유해 여러 개의 `refresh`를 피함.
- 첫 401이 `refreshTokens()` 호출하면, 나머지 401들은 같은 `promise`를 `await`함.

Step 3. refresh 호출

- `authSlice.refreshTokens()` → `/auth/refresh` (`refreshToken` 포함) 를 통해 새 `access` 발급하여 `store`에 세팅함.
- `verify_token(token, "refresh")`가 `type` 체크하고, `access` 토큰을 `refresh`로 사용 시도 같은 공격 차단.

Step 4. 원본 요청 재시도

- `originalRequest._retry = true` 마킹을 통해 무한 루프를 방지하고, 새 `access` 헤더를 주입하여 `api(originalRequest)` 재호출.

- refresh 실패 시 clearAuth() 함수 호출하여 로그아웃 화면으로 강제 이동.

1.9. 기타

1.9.1 AI 활용 상세

A. KoELECTRA: 일기 감정 분류

항목	내용
베이스 모델	monologg/koelectra-base-v3-discriminator (한국어 ELECTRA, ~110M 파라미터)
파인튜닝 라벨	8 클래스: joy / calm / proud / hope / sadness / anger / anxiety / fatigue (POSITIVE 4 + NEGATIVE 4)
배포	HuggingFace Inference API (V1 무료 티어, cold start 시 약 20초 — services/emotionAnalysis.ts 가 graceful fallback)
입력	일기 textContent (최대 약 500자)
출력	{tags:[], scores: {emotion: confidence}, sentiment: -1~1}
보조 신호	feedback/keywords.py 의 키워드 사전(8 라벨 × 30+ 표현, 구어체·신조어 포함). model 70% + keyword 30% 가중평균으로 confidence 보정.
활용 위치	클라이언트가 일기 작성 시점에 1회 호출 → 결과를 일기와 함께 sync. 서버는 추가 호출 없이 저장된 distribution을 리포트 집계에서 재사용.

B. Gemini Flash: 동적 피드백 생성

항목	내용
엔드포인트	https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent
호출 위치	services/gemini.py 의 두 함수 — enrich_diagnosis()(진단 detail 풍부화), generate_feedback()(본문 줄글 동적 생성)
generationConfig	temperature 0.7 (자연스러움 ↔ 일관성 균형), topP 0.9, maxOutputTokens 200~250 (긴 줄글 방지)
호출 트리거	exc.triggered(3 OR 패턴) OR selection["needs_dynamic"] (감정 3개 이상 동시) OR satisfaction.is_low (avg < -0.3) — 정적 템플릿이 약한 케이스에만 동적 생성
프롬프트 구조	(1) 역할 선언 ("HABITS 앱의 피드백 작성 어시스턴트") (2) 규칙 5개 (관찰적 표현, 처방 X, SDT 자율성, 2~3문장, 이모지 1개 이내) (3) 동적 톤 가이드 (만족도 hint) (4) 데이터(period_label, rate%, KoELECTRA 분포 JSON, 예외 reasons) (5) 작성 지시
폴백	API 키 미설정 → None 반환. timeout 15초. requests 예외 → None. 호출부에서 정적 narrative 그대로 사용.

품질 검사	<code>feedback/quality.py::check_gemini_output()</code> — 길이 20~250자, 금지 표현 8종(해야 합니다/반드시/꼭/즉시...), 인과 단정 4종(때문에/로 인해/탓에/원인), 따옴표 ≤ 4 , 이모지 ≤ 2 . <code>score = 1.0</code> - 패널티 + 관찰적 표현 가산점. <code>passed = score $\geq$ 0.7 AND 금지표현 없음 AND 인과 단정 없음</code> . fail 시 <code>static_fallback</code> 으로 marker 남기고 정적 detail 사용.
-------	---

C. Recombee: 협업 필터링 추천

항목	내용
SDK	<code>recombee_api_client</code> (Python)
DB	Production DB / region AP_SE / Private Token
시그널 매핑	<code>view</code> → <code>AddDetailView</code> (노출), <code>bookmark</code> → <code>AddBookmark</code> (수락), <code>purchase</code> → <code>AddPurchase</code> (이행), <code>AddRating</code> (-1~+1) — 만족도 명시 평가
활용 위치	새 습관 추천 (<code>/recommend/items</code>)과 만족도 신호 전송. 리포트의 "추천 행동" 라벨은 Recombee가 아니라 <code>narrative.py</code> 가 결정 (도메인 특화 규칙)
폴백	키 미설정 시 <code>_is_configured()</code> =False → 모든 함수 빈 결과 반환. SDK lazy import로 미설정 환경에서 <code>import cost</code> 없음.

D. 의사결정 분리: Gemini와 Recombee가 담당

- 데이터·구조는 백엔드 결정적 로직이 만들고 Gemini는 표현만 다듬기에, 환각 위험 최소화·결정 가능성을 보장함.
- Recombee는 사용자×아이템 행렬 학습을 함. 협업 필터링은 cold-start에 약하므로 보조 신호로만 사용하고, 메인 추천은 `narrative.py`의 통계 룰이 담당.

1.9.2. 인프라 배포

- 호스팅: Oracle Cloud
- DB: Supabase (PostgreSQL, Session Pooler 모드)
- DNS·TLS: DuckDNS, Caddy
- 메일: Resend를 통해 비밀번호 재설정 창구 마련
- 앱 상태: 백엔드는 단일 워커(`uvicorn --workers 1`)

1.9.3. 외부 라이브러리·버전

영역	라이브러리·버전
백엔드 웹	FastAPI 0.104.1, uvicorn
ORM·마이그레이션	SQLAlchemy 2.0.23, Alembic 1.12.1
인증	passlib[bcrypt] 1.7.4 + bcrypt 4.0.1 (passlib 호환), python-jose[cryptography]
스케줄러	apscheduler 3.10.4, pytz
HTTP	requests (Gemini), recombee_api_client (Recombee SDK), axios

프론트엔드	React Native 0.84.1, TypeScript
상태 관리	zustand + persist + @react-native-async-storage/async-storage
네비게이션	@react-navigation/native, native-stack

1.9.4. 보안·프라이버시 관련

A. 비밀번호 저장 — bcrypt cost 12

```
# backend/app/auth/password.py
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto", bcrypt__rounds=12)
```

항목	값·근거
알고리즘	bcrypt (Blowfish 기반 KDF, 64-byte salt 자동 생성, hash에 prefix)
cost factor	12 (= 2^12 라운드). 1회 해시 약 250ms (M1 기준). 단일 코어 brute-force 시 100K 시도/일 미만 → 일반적인 dictionary attack 대비 충분.
salt	bcrypt가 라운드별 자동 생성 → rainbow table 무력화. 같은 비밀번호도 매번 다른 hash.
평문 저장	DB의 어떤 컬럼에도 평문 비밀번호 저장 X. 로그·예외 메시지에도 비밀번호 echo 금지.
호환성 핀	bcrypt==4.0.1로 핀 — bcrypt 5.0 + passlib 1.7.4는 __about__ 속성 mismatch로 AttributeError 발생 (검증 단계에서 발견·수정).

B. 토큰 — JWT (HS256) 다중 타입 분리

```
# backend/app/auth/jwt_handler.py
def _create_token(subject, token_type, expires_delta):
    payload = {"sub": subject, "type": token_type,
               "iat": datetime.now(timezone.utc),
               "exp": datetime.now(timezone.utc) + expires_delta}
    return jwt.encode(payload, settings.SECRET_KEY, algorithm="HS256")
```

토큰 종류	만료 기간	용도	저장 위치 (클라이언트)
access	15분	API 인증	메모리 (zustand store, 새롭게 앱 부팅 시 휘발됨)
refresh	7일	access 갱신	AsyncStorage (zustand persist의 partialize로 refresh만 디스크에 진행)
reset	1시간	비밀번호 재설정	1회용 이메일 링크 전송

타입 분리의 의미는 다음과 같다.

- verify_token(token, expected_type="access")가 payload의 type 필드를 strict 비교하며, refresh 토큰을 access로 위조해 사용하는 공격 방어.

- **access** 토큰을 짧게(15분) 유지해 탈취 시 노출 시간 최소화. 사용자 경험 손상은 자동 **refresh**로 보완.
- **reset** 토큰의 **subject**는 **user_id**가 아니라 **email**으로, 토큰 자체로 계정 식별이 가능해야 비번 변경 **endpoint**에서 **user lookup** 가능.

Refresh race lock (frontend/src/services/api.ts)

동시에 여러 401이 발생할 때 **refresh**를 1회만 호출하도록 **single shared promise** 사용한다.

```
let refreshPromise: Promise<boolean> | null = null;
// ...
if (!refreshPromise) {
  refreshPromise = getStore().refreshTokens().finally(() => {
    refreshPromise = null;
  });
}
const refreshed = await refreshPromise; // 모든 401 요청이 같은 promise await
```

알고리즘을 요약하자면, 처음으로 들어온 **401** 신호에 대해 **refreshPromise**를 생성하여 **refresh**를 호출한다. 그 사이에 들어온 **N**개의 **401** 신호에 대해 같은 **promise**를 대기시켜 중복 호출이 되지 않도록 한다. 성공 시 새 **access**로 모든 대기 중인 요청을 재시도하며, **originalRequest._retry = true** 마킹을 통하여 리프레시 후에도 **401**이면 무한 루프에 빠지지 않도록 유의하여 과정을 진행한다. **refresh** 자체가 **401** 신호를 띄고 있다면, **clearAuth()** 함수를 호출하여 강제 로그아웃을 진행한다.

C. SECRET_KEY 관리

항목	정책
생성	<code>python -c "import secrets; print(secrets.token_urlsafe(64))"</code> — 64-byte URL-safe random
저장	.env 파일 (gitignore). 배포 환경은 Oracle Cloud 인스턴스의 환경변수
회전 정책	90일마다 회전하며, 회전 시점에 모든 refresh 토큰 invalidate (사용자 재로그인을 강제함)
노출 방지	FastAPI exception handler에서 <code>settings.SECRET_KEY</code> 등 환경변수 접근 결과를 <code>traceback</code> 에 echo 금지 (Pydantic settings의 <code>repr</code> 이 비밀 필드를 마스킹)

D. HTTPS, 전송 보안

항목	내용
TLS	배포 시 Caddy가 Let's Encrypt 인증서 자동 발급·갱신 (ACME HTTP-01) → 최소 TLS 1.2
HSTS	Caddy 기본값으로 <code>Strict-Transport-Security: max-age=31536000</code> 응답 헤더
CORS	FastAPI CORS 미들웨어로 모바일 앱 origin 만 허용. V2에서 웹 클라이언트 추가 시 화이트리스트 확장

E. SQL Injection, 입력 검증

방어선	구현
ORM 쿼리	모든 쿼리가 SQLAlchemy ORM (parameterized). raw SQL은 <code>text()</code> 래핑과 <code>bound param</code> 만 사용, <code>string interpolation</code> 비허용
입력 검증	모든 endpoint의 request body가 Pydantic schema로 통과. <code>EmailStr</code> , <code>constr(min_length=N)</code> , <code>conint(ge=0, le=100)</code> 등 도메인 제약을 schema 단계에서 강제
FK 무결성	<code>ondelete="CASCADE"</code> , <code>passive_deletes=True</code> 전역 적용, 사용자 삭제 시 종속 데이터 누락 없이 일괄 삭제

F. Authorization 패턴

<pre># 모든 사용자 데이터 endpoint 공통 r = db.query(AIReport).filter(AIReport.id == report_id, AIReport.user_id == current_user.id # ← 행 단위 소유권 검사).first() if not r: raise HTTPException(404, "...")</pre>
--

사용된 알고리즘은 다음과 같다.

- `get_current_user` 의존성이 토큰에서 `user_id` 추출하면, DB를 조회함.
- 단순 `Habit.id == habit_id`만으로 조회하지 않고, 다른 사용자의 ID를 알아도 `owner mismatch`로 판단함.

G. 외부 API 키 관리

키	저장	노출 방어
GEMINI_API_KEY	.env only	<code>services/gemini.py</code> 가 <code>query param</code> 으로만 전송, 로그에 <code>echo X</code>
HF_API_TOKEN	.env only	Authorization 헤더로만 전송. V2에서 self-hosted 모델로 이전 시 키 자체 제거 검토
RECOMBEE_API_TOKEN	.env only / private token 사용	Public Token 사용 시 HMAC mismatch — 검증 단계에서 발견·교체
RESEND_API_KEY	.env only	server-side만 사용

키 누락 시 `graceful fallback` 패턴 일관 유지: 각 서비스 모듈의 `_is_configured()`가 `False`면 `stub/None` 반환하여 프로덕션 키 누락이 500 에러로 번지지 않도록 설계했다.

H. 일기 텍스트 외부 전송

사항	내용
KoELECTRA 호출	HF Inference API로 일기 원문 전송
Gemini 호출	일기 원문은 미전송 (집계된 <code>distribution</code> , 라벨만 전송)
데이터 마스킹	사용자 개인정보(이름·이메일)는 KoELECTRA 입력에 포함되지 않고, 일기 본문만 포함됨

원본 텍스트가 외부 API로 나가는 경로는 KoELECTRA 1곳으로 Gemini는 통계 메타만 받도록 설계했다. (feedback/diagnosis.py::category_summary 형식의 dict)

I. 계정 삭제

User 삭제

- ondelete CASCADE
- personality_results (1:1)
- schedules (1:1)
- habits (1:N)
 - habit_logs (1:N) [habits에 cascade]
- diaries (1:N)
 - emotion_analyses (1:1) [diary에 cascade]
- ai_reports (1:N)
- feedback_ratings (1:N)
- recommendation_logs (1:N)

알고리즘에 대한 설명은 다음과 같다.

- 모든 sync target과 종속 테이블에 ForeignKey(..., ondelete="CASCADE"), relationship에 passive_deletes=True를 통해 단일 DELETE 문이 DB 레벨 cascade로 일괄 처리. 애플리케이션이 row-by-row 삭제하지 않음.
- 검증 단계에서 발견된 누락(HabitLog cascade 누락 → NotNullViolation)을 수정. 모든 테이블에 동일 정책 적용.
- soft delete(deleted_at 컬럼)는 sync용 tombstone 목적으로만 사용. 계정 삭제는 hard delete로 처리함.

J. 로그 정책

항목	정책
비밀번호	어떤 경로(stdout/file/exception)에서도 echo 금지
JWT 토큰	exception 메시지에 토큰 echo 금지 (verify_token이 detail에만 메시지 노출, 토큰 본문은 미포함)
일기 원문	서버 로그에 일기 본문 출력 금지. KoELECTRA 호출 실패 시 에러 메시지만 로그 ([report] user {id} weekly 실패: {e} 같은 형태로, 사용자 ID와 예외 타입만)
HuggingFace/Gemini 응답	디버그 모드에서만 prompt-response 로그로 남기며, 프로덕션은 metadata(latency, status code)만을 취급.

2. 과제 설계

2.1. 요구사항 정의

기능적 요구사항

본 서비스의 요구사항은 서비스 흐름에 따라 온보딩, 습관 이행 관리, 감정 일기, 피드백, 마이페이지의 5개 도메인으로 나뉘며, 각 도메인별로 기능적 요구사항을 정의하였다. 우선순위는 서비스의 핵심 가치 제안과의 연관성 및 기술적 의존 관계를 고려하여 상(필요)·중(중요)·하(개선) 3단계로 분류하였다.

ID	도메인	요구사항	상세 설명	우선순위	진척도
FR-01	온보딩	성격 유형 진단	자기결정이론 기반 문항 응답을 통해 사용자의 현재 성격 유형과 이상적 성격 유형 총 2가지를 도출한다.	상	완료
FR-02	온보딩	성격 특성 해시태그 선택	도출된 성격 유형에 할당된 해시태그 중 사용자가 강화를 희망하는 특성을 복수 선택할 수 있어야 한다.	상	완료
FR-03	온보딩	일상 루틴 입력	통근, 식사, 업무/수업 등 사용자의 일상 시간대 정보를 입력받아 추천 맥락 데이터로 활용한다.	상	완료
FR-04	온보딩	맞춤 습관 추천	선택된 해시태그와 입력된 루틴을 기반으로, 124개 습관 템플릿에서 적합도 점수를 산출하여 정렬된 추천 목록을 제시한다.	상	정규화 수정 중
FR-05	온보딩	습관 일정 설정	사용자의 일상 패턴을 반영한 반복 요일 및 시각을 자동 제안하며, 사용자가 수동으로 조절할 수 있어야 한다.	상	완료
FR-06	습관 관리	일별 이행 기록	설정된 습관 일정에 따라 체크리스트를 자동 생성하고, 사용자의 당일 이행 현황을 기록한다.	상	완료
FR-07	습관 관리	이행률 집계	주간/월간/연간 단위로 전체 해시태그별 습관 달성률을 자동 산출하여 요약 제공한다.	상	완료
FR-08	습관 관리	감편 감정 입력	기분 점수 슬라이더(0~100)와 감정 키워드 버튼('기쁨', '슬픔', '피로' 등)을 통해 감정을 간편하게 기록할 수 있어야 한다.	상	완료
FR-09	감정 일기	텍스트 일기 및 감정 분석	사용자가 선택적으로 작성한 텍스트 일기를 KoELECTRA 모델에 입력하여, 감정 레이블 및 확률 분포를 산출하고 DB에 저장한다.	상	파인튜닝 진행 중
FR-10	피드백	달성률 피드백	달성률 구간(목표: 10% 단위)에 따라 로컬 폴백 기반의 프로그레시브 메시지를 제공한다.	중	부분 구현 (현재 30%, 70% 기준으로 피드백 진행 중)
FR-11	피드백	감정-이행	감정 분석 결과와 습관 이행률을 교차 분석하여,	중	구현 중

		를 교차 분석 피드백	감정 변화 추이 그래프 및 상관관계 기반 피드백 메시지를 생성한다.		
FR-12	피드백	AI 피드백 리포트 생성	Gemini Flash API (동적)와 로컬 폴백(정적)을 결합한 하이브리드 방식으로, 주/월간 개인화 피드백 리포트를 생성한다.	중	구현 중
FR-13	피드백	습관 강도 자동 조정 제안	이행률 30% 미만 시 빈도 축소/습관 변경, 90% 이상 시 빈도 증가/습관 추가/습관 졸업을 자동으로 제안한다.	중	구현 예정
FR-14	피드백	일정 및 습관 관리	일상 루틴 수정, 해시태그 재선택, 습관 교체 시 변경 사항이 체크리스트 및 추천 로직에 정상 반영되어야 한다.	중	부분 구현
FR-15	피드백	알림 설정	습관 수행 알림 및 피드백 리포트 도착 알림의 시간·빈도를 사용자가 설정할 수 있어야 한다.	하	구현 예정

비기능적 요구사항

비기능적 요구사항으로는 **API 응답 시간 2초 이내의 응답 속도**, **Gemini API 호출 최소화**를 위한 로컬 폴백 우선 구조의 비용 효율성, 동적 생성 메시지와 정적 메시지 간 어조의 일관성 유지(이는 그로스 학기 지도교수 피드백에서도 강조된 사항이다), **React Native** 기반 **iOS/Android** 동시 지원을 위한 크로스 플랫폼 호환성, 그리고 사용자 감정 일기 등 민감 데이터의 암호화 저장을 위한 데이터 보안이 정의되어 있다.

ID	분류	요구사항	상세설명	목표 수치
NFR-01	성능	API 응답 속도	감정 분석 및 피드백 생성을 포함한 모든 API 호출의 사용자 체감 응답 시간을 목표 이내로 유지한다.	≤ 2초
NFR-02	비용 효율성	LLM 호출 비율 제한	전체 피드백 메시지 중 Gemini Flash API 를 통한 동적 생성 비율을 제한하여, 월간 API 비용을 통제한다. 나머지는 로컬 폴백으로 처리한다.	동적 생성 ≤ 30%
NFR-03	사용자 경험	메시지 톤 일관성	Gemini 가 생성하는 동적 코칭 메시지와 로컬 폴백 정적 메시지 간의 어조·어투가 사용자에게 이질감 없이 일관되게 유지되어야 한다. 프롬프트 가이드라인을 통해 확보한다.	베타 테스트 이질감 보고율 ≤ 10%
NFR-04	호환성	크로스 플랫폼 지원	React Native 기반으로 iOS 및 Android 환경에서 동일한 기능과 UI/UX 를 제공해야 한다.	iOS 14+ / Android 10+
NFR-05	보안	민감 데이터 보호	사용자의 감정 일기 텍스트, 성격 유형 결과 등 민감한 개인 데이터는 전송 시 TLS 암호화, 저장 시 AES-256 암호화를 적용한다.	전송·저장 시 암호화 적용

NFR-06	확장성	습관 템플릿 확장	서비스 운영 중 습관 템플릿 및 해시태그를 추가·수정할 수 있도록 데이터 모델을 유연하게 설계한다.	템플릿 추가 시 코드 변경 불필요
NFR-07	가용성	오프라인 대응	네트워크 미연결 상태에서도 습관 체크리스트 기록과 감정 슬라이더 입력이 가능하도록 로컬 캐싱을 지원하고, 연결 복구 시 자동 동기화한다.	핵심 기록 기능 오프라인 동작
NFR-08	유지보수성	모듈화 구조	AI 엔진(감정 분석, 피드백 생성, 추천)을 독립 모듈로 분리하여, 개별 엔진의 교체·업그레이드가 서비스 전체에 영향을 주지 않도록 설계한다.	엔진별 독립 배포 가능

2.2. 상세 기능 명세

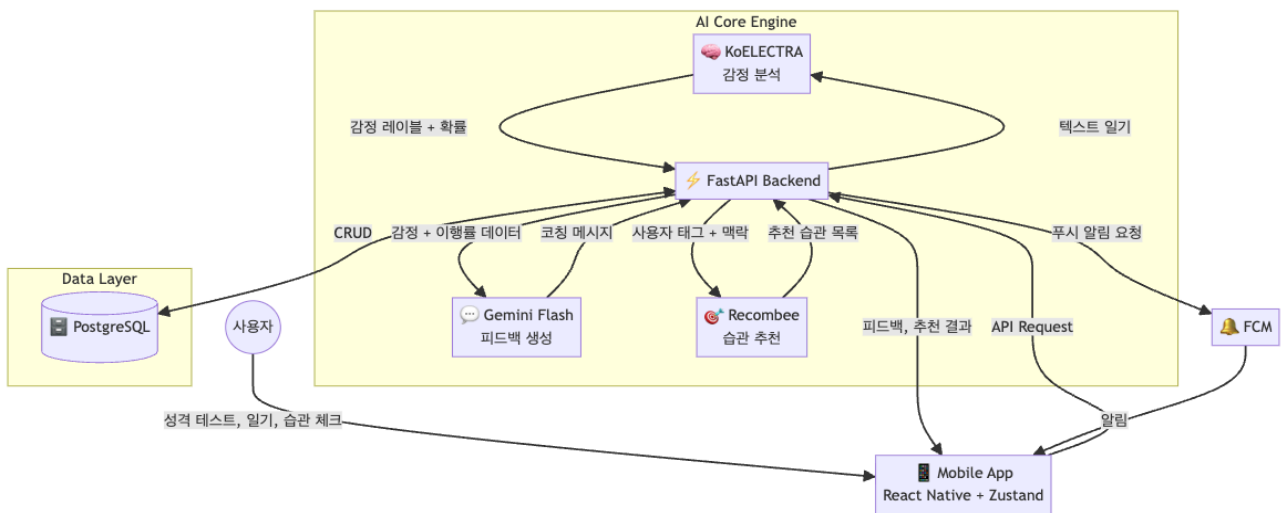
구분	상세 기능	기능 설명	상태
온보딩	성격 특성 진단	자기결정론 기반 30개 문항을 통한 성격 테스트 진행 및 ‘현재의 성격’, ‘이상적 자아’ 결과 도출	구현 완료
습관 추천	태그 기반 습관 매칭	80개 성격 해시태그와 124개 습관 템플릿을 매핑하여 사용자 성향에 최적화된 습관 후보 리스트 도출	조정 중 (습관 템플릿 개수, 태그 수)
습관 추천	상황 인식 스케줄링	사용자의 가용 시간과 습관의 특성을 고려하여 개인화된 일일 루틴 자동 배치	고도화 중
감정 일기	감정 데이터 수집	슬라이더 및 텍스트 입력을 통해 일기 작성 시 실시간 감정 상태 (기쁨, 슬픔, 피로 등) 데이터 저장	구현 중, 텍스트 분석 모델 파인튜닝 진행 중
분석	감정-습관 이행을 교차 분석	습관 이행률과 기록된 감정 수치 간의 상관관계를 계산하여 주간/월간 통계 대시보드를 시각화	구현 중
코칭	AI 적응형 피드백	Gemini Flash API 를 연동하여 사용자의 습관 이행 패턴과 감정 상태에 맞춘 개인화된 동기부여 메시지 생성	구현 예정

2.3. 전체 시스템 구성

시스템 아키텍처 개요

HABITS는 3-Tier 아키텍처 기반으로 설계되었다. 프레젠테이션 계층(React Native), 비즈니스 로직 계층(FastAPI), 데이터 계층(PostgreSQL)이 명확히 분리되어 있으며, AI 엔진들은 비즈니스 로직 계층의 독립 모듈로서 외부 API 또는 내부 모델 서빙을 통해 연동된다.

사용자의 입력은 모바일 앱을 통해 FastAPI 서버로 전송된다. 서버는 요청의 성격에 따라 세 가지 AI 엔진에 데이터를 분배한다. 텍스트 일기는 KoELECTRA 감정 분석 엔진에 전달되어 감정 레이블과 확률 분포를 반환받고, 감정 분석 결과와 이행을 데이터는 Gemini Flash 피드백 생성 엔진에 전달되어 맞춤 코칭 메시지를 생성하며, 사용자의 성격 태그와 맥락 정보는 Recombee 추천 엔진에 전달되어 적합 습관 목록을 반환받는다. 모든 원천 데이터와 분석 결과는 PostgreSQL에 영속 저장되며, 생성된 피드백과 알림은 Firebase Cloud Messaging을 통해 사용자 단말로 비동기 전송된다.



기술 스택과 선정 사유

계층	기술	버전/모델	선정 사유
Frontend	React Native	0.73+	iOS/Android 크로스 플랫폼 단일 코드베이스 개발
Backend	FastAPI (Python)	0.100+	비동기 처리 최적화, AI 라이브러리(transformers, httpx 등) 호환
Database	PostgreSQL	15+	JSONB 타입 지원으로 사용자 맥락 데이터의 유연한 스키마 운용
상태 관리	Zustand + AsyncStorage	-	경량 상태관리 및 로컬 영속 저장, 오프라인 캐싱 지원
감정 분석	KoELECTRA	monologg/koelectra-base-v3-discriminator	Google ELECTRA 아키텍처를 한국어 코퍼스로 사전 학습한 경량 모델로, 적은 GPU 리소스로도 도메인 특화 파인튜닝이 가능
피드백 생성	Gemini Flash API	gemini-2.0-flash	기존 사용 예정이었던 GPT-4o 대비 비용 절감 및 빠른 응답, 한국어 성능 우수
추천 엔진	Recombee	-	콜드스타트 대응 가능, 아이템 기반 협업 필터링 내장
푸시 알림	Firebase Cloud Messaging	-	크로스 플랫폼 푸시 알림, 딥링크 연동 지원

기술 스택 관련 엔진 검증

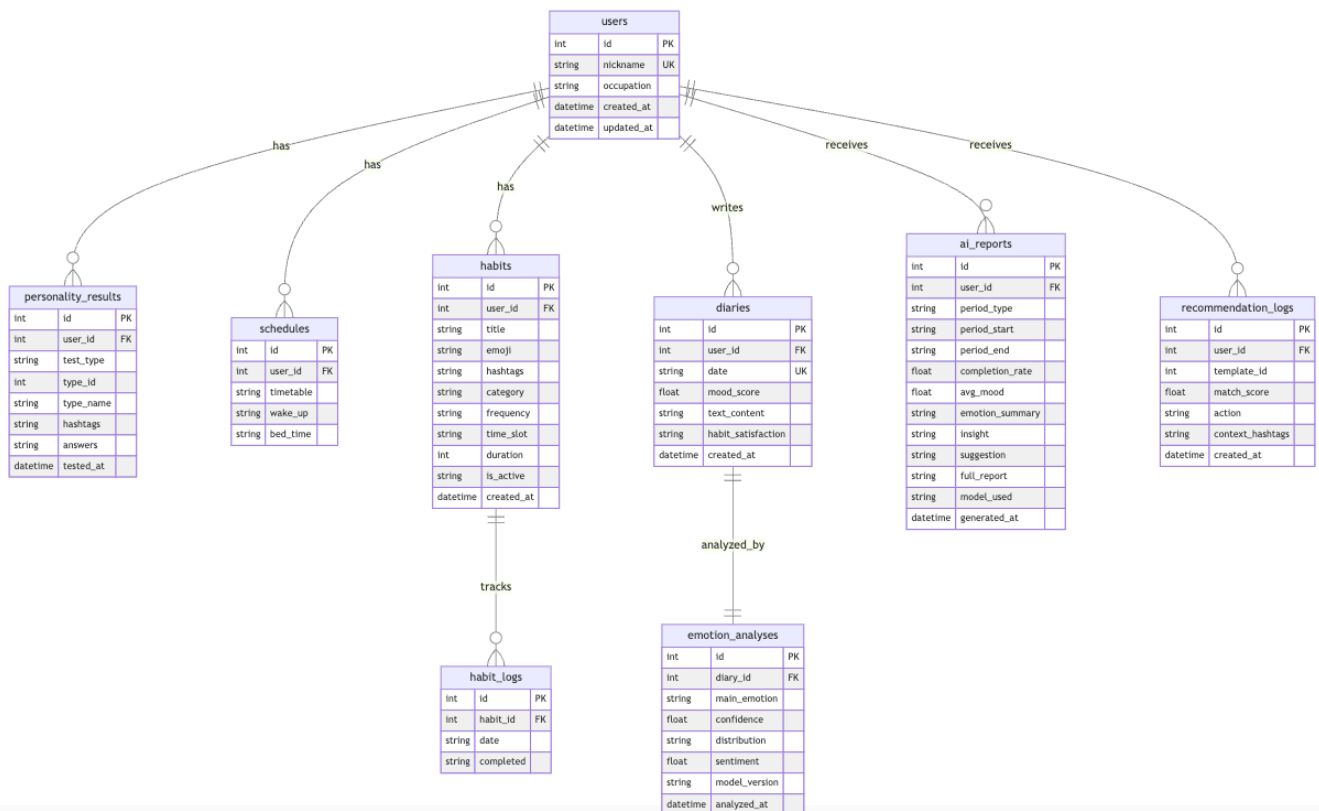
NLP 엔진인 KoELECTRA의 경우, 일기 데이터의 감정 분류를 위해 KoBERT와 KoELECTRA를 비교 검증하였다. 한국어 구어체 데이터셋 테스트 결과, KoELECTRA가 문맥 파악 속도와 복합 감정 분류에서 약 15% 높은 F1-Score를 기록하여 최종 엔진으로 채택하게 되었다.

LLM 서비스는 실시간 코칭 메시지 생성을 위해 모델별 지연 시간(Latency)을 측정하였다. 타 모델 대비 30% 빠른 응답 속도를 확인하였으며, 긴 문맥 유지 성능이 우수하여 사용자의 과거 습관 이력을 반영한 일관된 코칭이 가능함을 검증하였다.

데이터베이스 스키마 설계

HABITS의 데이터베이스는 개인화된 코칭을 위해 사용자 맥락과 행동 로그를 유기적으로 연결하는 관계형 구조로 설계되었으며, 단순한 습관 저장을 넘어 추천 엔진과 피드백 루프를 지원하기 위해 5가지 핵심 엔티티를 정의하였다.

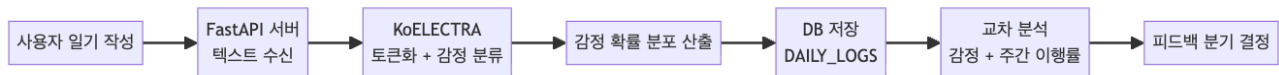
USERS 테이블은 사용자의 기본 정보뿐만 아니라, 이상적 자아 태그(예: #대담함, #도전)와 라이프스타일 맥락(통근 시간, 기상 시간 등)을 **JSONB** 형태로 저장하여 유연한 개인화 추천을 지원한다. **HABIT_TEMPLATES** 테이블은 **Recombee** 추천 시스템의 아이템 기반이 되는 124개의 표준 습관 데이터를 관리하며, 각 습관의 카테고리, 난이도, 연결된 해시태그 정보를 포함한다. **USER_HABITS** 테이블은 템플릿을 기반으로 사용자가 자신의 상황에 맞춰 빈도와 알림 시간을 커스터마이징한 실행 데이터를 관리한다. **DAILY_LOGS** 테이블은 매일의 습관 이행 여부와 일기 텍스트, 감정 슬라이더 점수, 그리고 **KoELECTRA** 모델이 분석한 감정 결과를 통합 저장하여 **AI** 피드백의 기초 데이터로 활용된다. **WEEKLY_REPORTS** 테이블은 주간 단위의 달성률과 감정 데이터를 집계하고, **Gemini** 혹은 로컬 폴백이 생성한 피드백 스냅샷을 저장하여 사용자의 성장을 시각화하는 데 활용된다.



<서비스의 데이터베이스 ERD>

AI 파이프라인 상세: 감정 분석 흐름

사용자가 일기 쓰기 기능을 통해 텍스트를 작성하면, 해당 데이터는 **FastAPI** 백엔드 서버로 전송되어 **KoELECTRA** 모델의 입력값이 된다. 모델은 입력된 문장을 토큰화하고 분석하여, 해당 문장에 내포된 감정의 확률 분포(예: 슬픔 **85%**, 불안 **10%**)를 산출한다. 이후 시스템은 이 감정 분석 결과와 사용자의 주간 습관 이행을 데이터를 결합하여 피드백 분기를 결정한다. 예를 들어, 사용자의 습관 이행률이 저조하고 감정이 '슬픔'으로 분석되었다면, 시스템은 사용자를 질책하지 않고 위로의 메시지를 전달하며 습관의 빈도나 강도를 줄일 것을 부드럽게 제안하는 방향으로 피드백을 생성한다.



AI 파이프라인 상세: 피드백 생성 하이브리드 구조

피드백 생성 시 모든 메시지를 **LLM**으로 생성하면 비용과 속도, 안정성 측면에서 문제가 발생할 수 있다는 점은 스타트 학기 지도교수 면담에서도 지적된 바 있다. 이에 본 팀은 자주 발생하는 상황에 대해서는 사전 정의된 메시지 풀에서 적절한 정적 메시지를 선택하여 제공하고(로컬 풀백), 복합적 감정 패턴이나 급격한 이행률 변화 등 특수 상황이 발생한 경우에만 **Gemini Flash API**를 호출하여 사용자의 구체적 데이터를 프롬프트에 주입한 동적 코칭 메시지를 생성하는 하이브리드 구조를 채택하였다. 로컬 풀백의 메시지 풀은 사용자가 틀에 박힌 응답이라는 인상을 받지 않도록 충분히 다양화할 예정이며, **Gemini API** 호출 시에는 시스템 프롬프트를 통해 자기결정이론 기반의 공감적 습관 코치 '하빗' 페르소나를 부여하여 어조와 톤의 일관성을 확보한다. 동적 메시지와 정적 메시지 간의 톤 일관성 유지는 그로쓰 학기 지도교수 피드백에서도 강조된 사항으로, 프롬프트 엔지니어링 단계에서 별도의 어조 가이드라인을 수립하여 대응할 계획이다.

2.4. 주요 엔진 및 기능 설계

엔진 1: KoELECTRA 감정 분석 엔진

사용자가 텍스트 일기를 작성하면, FastAPI 서버가 이를 KoELECTRA(monologg/koelectra-base-v3-discriminator) 모델에 전달한다. 모델은 입력 문장을 WordPiece 토큰나이저로 토큰화한 뒤, 감정 분류 헤드를 통해 감정별 확률 분포를 산출하며, 결과는 DAILY_LOGS 테이블의 koelectra_result 컬럼에 JSONB 형태로 저장된다. 현재 한국어 구어체 일기에 대한 정확도를 높이기 위해 초기 파인튜닝을 진행 중이며, 최종 목표 정확도는 85% 이상이다.

예상 결과: 사용자가 "오늘 하루 계획대로 된 게 없어서 무기력하다"라고 입력하면, 모델이 {sadness: 0.85, anxiety: 0.10, anger: 0.03, joy: 0.02}와 같은 확률 분포를 반환하고, 이 결과가 주간 이행을(예: 45%)과 결합되어 "위로 + 습관 강도 하향" 방향의 피드백 분기를 결정한다.

AI Hub <감성 대화 말뭉치>를 베이스로 하여 감정 라벨 리매핑을 진행하고, 클래스 가중치 손실을 의도하였다. 이와 같은 작업을 진행하여 감정 키워드 사전을 보완하였다.

엔진 2: Gemini Flash 피드백 생성 엔진

스타트 학기 지도교수의 "템플릿 응답과 LLM 생성을 혼합하라"는 피드백을 반영하여 하이브리드 구조를 설계하였다. 자주 발생하는 상황(달성을 구간별 일반 패턴)에는 사전 정의된 정적 메시지 풀에서 랜덤 선택(로컬 풀백)하고, 특수 상황(복합 감정, 급격한 이행을 변화 등)에만 Gemini Flash API를 호출하여 사용자 데이터를 프롬프트 컨텍스트에 주입한 동적 코칭 메시지를 생성한다. 시스템 프롬프트에는 자기결정이론 기반 '하빗' 코치 페르소나를 부여하여, 사용자를 비난하지 않고 공감하며 구체적 행동을 권유하는 어조를 유지한다. Gemini 호출 비율은 전체 피드백의 30% 이내로 제한하여 월간 API 비용을 통제한다.

예상 결과: 이행을 40% + 감정 '슬픔' 조합에서, "이번 주는 많이 힘드셨죠? 무리하지 말고, 내일은 '5분 명상'처럼 가벼운 습관 하나만 해보는 건 어떨까요?"와 같은 개인화 코칭 메시지가 생성된다.

엔진 3: 습관 추천 로직

온보딩 시 사용자가 선택한 해시태그와 입력한 일상 루틴을 기반으로, 습관 템플릿 데이터베이스에서 적합 습관을 매칭한다. 맥락 일치 점수(context_match_score)를 산출하여 적합도 순으로 정렬하며, 사용자가 직관적으로 파악할 수 있도록 적합도를 퍼센티지로 시각화한다. 현재 적합도가 100%를 초과하는 Normalization 에러가 확인되었으며, 지도교수 피드백에 따라 각 인자값의 0~1 min-max 스케일링 후 Softmax 함수를 적용하여 해결할 계획이다. 또한 Hard Constraint만으로 필터링 시 조건 부합 습관이 누락되는 문제를 보완하기 위해, 일정에 맞지 않더라도 선호도가 높은 습관을 보조 추천으로 제안하는 Soft Constraint를 도입할 예정이다.

2.5. 개발 내용 및 현황

스타트 학기 성과 요약

스타트 학기에서는 서비스의 기획과 아키텍처 설계에 집중하였다. 심리학 기반 데이터 모델링을 완료하여 ~~80개~~30개 성격 특성 해시태그와 ~~424개~~300개 습관 템플릿을 설계하였으며, 3-Tier 아키텍처를 확립하고 **React Native**와 **FastAPI**를 연결하는 기본적인 **API** 규격을 설계하였다. 5개 핵심 엔티티를 중심으로 데이터베이스 스키마를 정의하였고, **KoELECTRA** 감정 분석 파이프라인의 개념 설계와 **LLM** 기반 피드백 시스템 프롬프트의 시뮬레이션 및 검증을 수행하였다. 기본 **UI/UX** 와이어프레임 역시 이 시기에 설계되었다. 다만 냉정히 평가하면, 기획 단계에 상당한 시간이 소요되어 실제 기술 구현은 스켈레톤 수준에 머물러 있었으며, 이 점은 지도교수 면담에서도 "기획의 깊이는 우수하나 엔지니어링 구현이 시급하다"는 피드백으로 확인된 바 있다.

그로쓰 학기 개발 현황

그로쓰 학기에서는 스타트 학기의 교훈을 바탕으로 실질적인 개발에 집중하였으며, 현재 아래와 같은 진척도를 보이고 있다.

구현이 완료된 항목으로는, **PostgreSQL** 기반 데이터베이스 마이그레이션, 사용자/습관/일기 관련 기본 **CRUD API** 구현, 성격 유형 테스트 진행 및 결과 도출 로직 구현, 해시태그 기반 습관 매칭 및 추천 기능 구현, 캘린더 기반 메인 페이지 **UI** 구현, 감정 슬라이더/감정 버튼/텍스트 입력이 포함된 감정 일기 **UI** 구현, 성격 테스트 **UI** 구현, 그리고 피드백 페이지의 달성도/감정 분석/일기 3개 탭 기본 **UI** 구현이 있다.

현재 진행 중인 항목으로는, 한국어 감정 분류 모델인 **KoELECTRA**의 초기 파인튜닝 작업, 이행을 따른 습관 강도 조정 제안 로직 개발, 감정 분석 결과와 이행을 데이터를 연계하는 교차 분석 로직 구현, 그리고 습관 추천 시 적합도 퍼센티지 표시 과정에서 발생하는 **Normalization** 에러의 수정이 있다. 또한 마이페이지에서 변경된 습관 일정이 정상적으로 반영되지 않는 문제도 수정 중에 있다.

향후 구현이 예정된 항목으로는, **Gemini Flash API** 연동을 통한 동적 코칭 메시지 생성, 피드백 리포트 내용 구성 및 생성 로직 전반, 달성을 구간별 메시지 세분화(현재 30%/70% 기준에서 10% 단위 분기로 확장), 일기 탭의 **UX** 개선(페이지네이션, 기간별 필터링, 탭 네비게이션), 마이페이지의 알림 설정 기능, 그리고 리포트에서 제안된 습관 조정을 실제 서비스에 반영하는 연동 로직이 있다.

현재 수정 중인 기술적 이슈

현재 개발 과정에서 확인된 주요 기술적 이슈는 세 가지이다.

첫째, 습관 추천 시 적합도가 100%를 초과하는 에러가 발생하는 문제이다. 이는 사용자의 일상 루틴과 성격 특성이라는 서로 다른 도메인의 데이터를 결합하는 과정에서 **Normalization**이 누락되었거나 가중치 합산에 오류가 있기 때문으로 추정된다. 그로쓰 학기 지도교수 피드백에 따라 각 인자값을 0~1 사이로 스케일링한 후 최종 **Softmax** 처리를 거치는 방식을 검토 중이다.

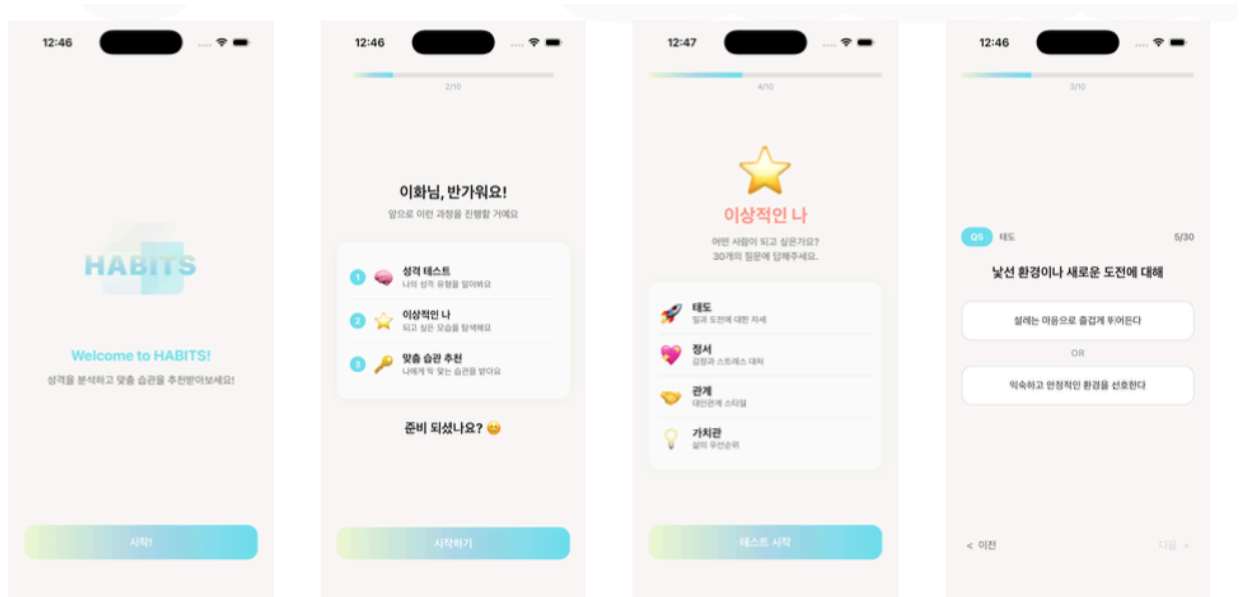
둘째, 사용자가 선택한 해시태그에 해당하면서 일정에도 부합하는 습관임에도 불구하고 추천 목록에서 누락되는 문제가 발생하고 있다. 이는 추천 모델의 **Hard Constraint** 설정이 과도하게 엄격하여, 일정에 정확히 맞는 습관만 필터링하는 구조에서 기인하는 것으로 분석된다.

지도교수의 조언에 따라, 일정에 맞지 않더라도 선호도가 높은 습관은 제안하고 사용자가 일정을 조정하도록 유도하는 **Soft Constraint**의 도입을 검토하고 있다.

셋째, 일기 작성 페이지에서 작성 직후 습관 적합성을 평가하여 바로 습관을 조정하는 기능이 존재했으나, 너무 잦은 확인과 변경이 오히려 사용성을 낮출 수 있다는 판단 하에 해당 기능을 제거하고, 주간 피드백 리포트 시점에 습관 조정을 통합 제안하는 방식으로 로직을 변경하고 있다.

UX/UI 현황

현재 개발 중인 UI의 일부이다. 각 스크린샷 아래에 있는 검정색 문구는 해당 화면에서 보여주는 기능들에 대한 간략한 설명이며, 보고서 작성 시점에서 아직 완벽하게 구현되지 않은 기능이나 구현되었으나 수정 중에 있는 화면들은 제외하였다.

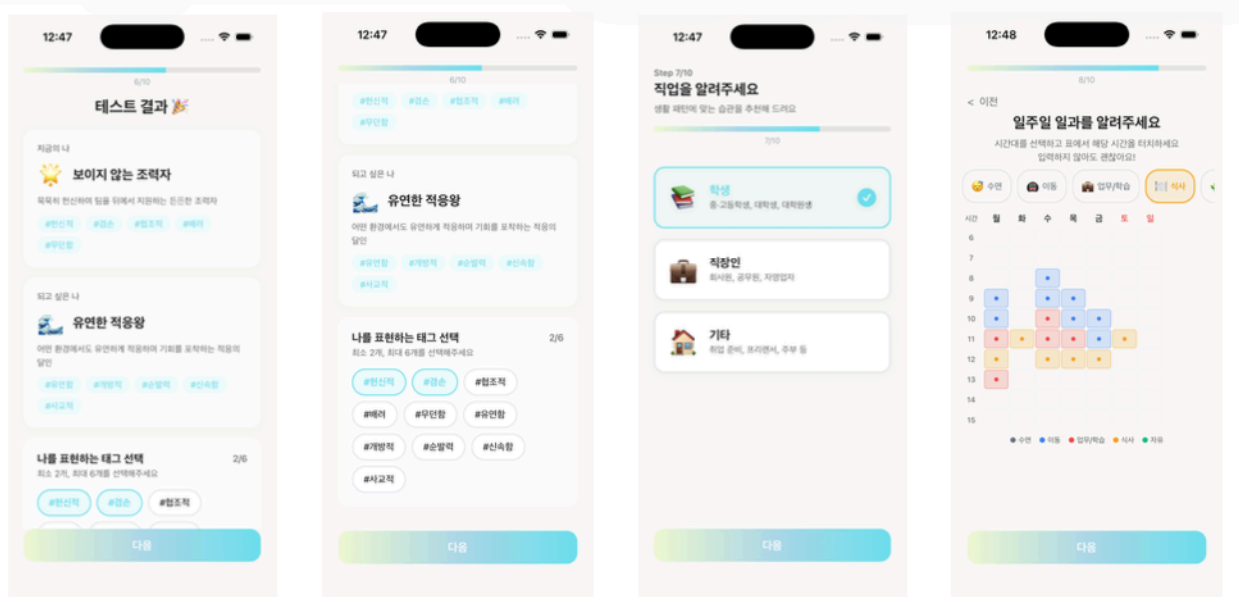


온보딩 화면

진행 순서 안내

테스트 영역 안내

현재 성격 & 이상적인 성격 테스트

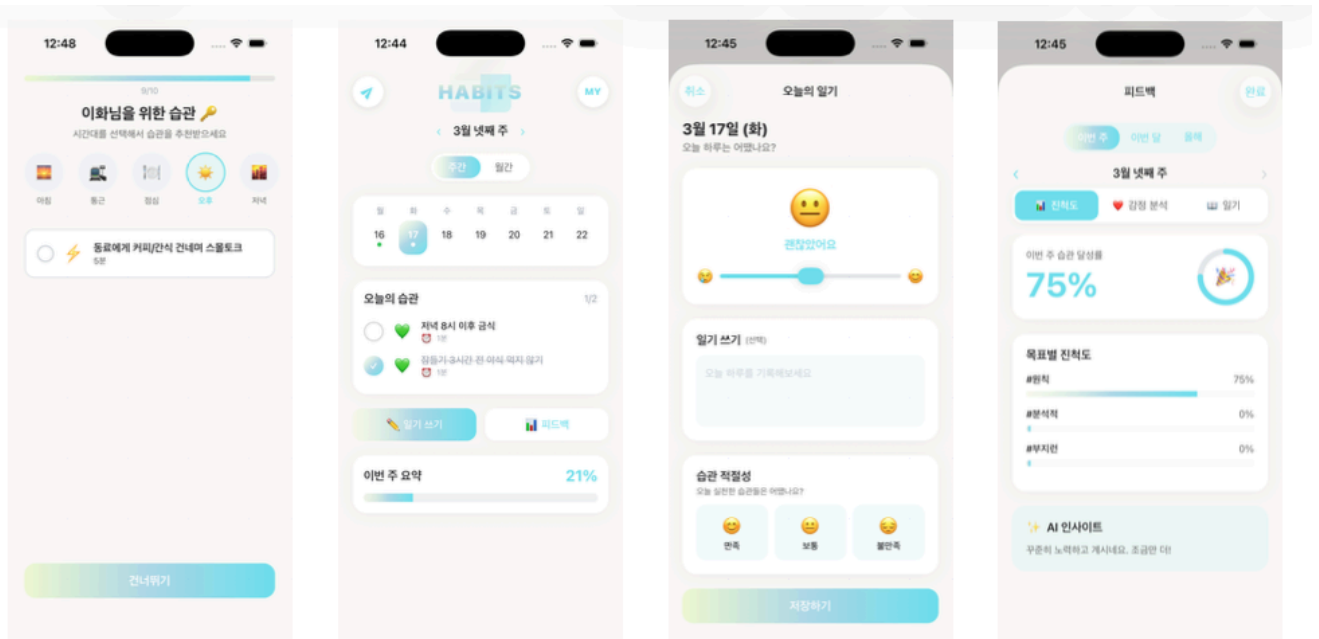


성격 테스트 결과

해시태그 선택

사용자 일정 파악 (직업)

사용자 일정 파악 (시간)



시간대별 습관 추천

메인 화면

일기 작성 페이지

피드백 페이지

3. 팀 정보

팀 구성 및 역할 분담

Team 12 HABITS는 양설아와 **Deng Yuanrong** 2인으로 구성되어 있다. 스타트 학기 동안 팀장인 양설아가 백엔드/AI를, 팀원인 **Deng Yuanrong**이 프론트엔드를 각각 전담하는 구조로 개발을 진행하고자 하였으나, 여러 시행착오를 겪었다. 그로쓰 학기 이후부터 두 팀원은 합의 하에 역할 분담 명시 내역을 더욱 사실적으로 수정하게 되었다.

또한 실질적 역할 분담에 대하여서는, 그로쓰 학기부터는 두 팀원이 실질적으로 책임감을 가지고 기여 가능한 부분에 참여하되, 각자의 강점에 따라 주요 담당 영역을 분배하는 방식으로 전환하였다. 이로 인해 보고서 제출 시점에서 확정된 팀원 각자의 역할은 다음과 같다.

양설아는 팀장으로서 풀스택 및 **AI** 영역을 주로 담당하고 있다. 구체적으로는 서비스 전반의 **UX/UI** 설계, 주요 **API** 설계, 각종 **AI API** 연동, **API** 테스트 및 시스템 기능 검증 수행, 습관·일기·감정 데이터 모델 설계 및 **API** 개발, **KoELECTRA** 파인튜닝 및 해당 모델 기반 감정 분석 기능 구현, **Gemini** 기반 공감 메시지 및 피드백 구조 구현, 습관 추천 알고리즘 구조 설계, 그리고 사용자 습관 이행을 분석 및 리포트 생성 기능 구현, 보고서 작성 및 일정 할당과 조정을 담당한다.

Deng Yuanrong은 팀원으로서 일정 및 태스크 리마인드 역할을 맡고 있다.

4. 향후 계획

그로쓰 학기 잔여 일정

보고서 작성 시점(3월 29일) 기준으로 그로쓰 학기의 잔여 일정은 다음과 같이 계획되어 있다.

기간	주요 과제	목표 산출물	정량 목표
4월	KoELECTRA 파인튜닝 완료, 습관/감정/피드백 API 추가 개발	감정 분석 API 동작 검증, 피드백 분기 로직 완성	감정 분류 정확도 $\geq$ 85%
5월	Gemini Flash 연동, 톤 일관성 검증, 포스터 리뷰	리포트 생성 기능 완성, 동적/정적 메시지 QA	Gemini 호출 비율 $\leq$ 30%
6월	전체 통합 테스트, 실사용자 베타 테스트, Oracle cloud를 통한 인프라 배포	베타 테스트 피드백 보고서, 최종 발표	사용자 만족도 $\geq$ 80%

베타 테스트는 타겟 사용자층인 20대 대학생 및 사회초년생을 대상으로 진행하며, 추천 적합성 만족도, 감정 분석 정확도, 피드백 공감도, 전반적 서비스 사용성을 중점으로 검증한다.

지도교수 및 담당교수 피드백과 반영 사항

본 프로젝트는 지도교수(심재형 교수님)와 담당교수(유광현 교수님) 두 분으로부터 지속적인 피드백을 받으며 개선을 진행해왔다.

담당교수인 유광현 교수님은 1차 과제 리뷰 미팅에서 두 가지 핵심 피드백을 주셨다. 첫째, 과제 제목이 타겟 커스트머와 Pain Point를 더 명확히 반영하도록 수정할 것을 제안하셨으며, 이를 반영하여 과제 제목을 현재의 형태로 수정 완료하였다. 둘째, 문제점과 해결 방안 사이에 논리적이고 객관적인 근거가 필요하다는 점을 지적하셨다. 이러한 피드백을 반영하여 자기결정이론(SDT), JITAI, Dijkstra & De Vries 연구 등 행동심리학 분야의 학술 논문을 조사하여 각 솔루션의 개연성을 확보하였다. 마지막으로, 세부 기능에 대한 충분한 설명이 부족하다는 피드백을 주셨다. 마지막 피드백을 반영하여 과제 수행 계획 발표 자료에서 세부 기능 페이지를 추가하는 방향으로 수정하였다.

지도교수인 심재형 교수님으로부터는 스타트 학기 최종 면담과 그로쓰 학기 이메일 면담을 통해 총 다섯 가지의 주요 피드백을 받았다. 스타트 학기에는 기획의 깊이는 우수하나 엔지니어링 구현이 시급하다는 점과, 모든 피드백에 LLM을 호출하면 비용과 속도 문제가 발생하므로 하이브리드 방식을 고려할 것을 제안받았다. 전자에 대해서는 그로쓰 학기 개발 일정을 집중 조정하여 대응하였고, 후자에 대해서는 로컬 풀백과 Gemini를 결합한 하이브리드 구조를 채택하여 반영하였다. 그로쓰 학기 이메일 면담에서는 사용자에게 습관 추천 시 습관 적합도가 100%를 초과하는 에러에 대해 각 인자값의 0~1 스케일링 후 Softmax 처리를 확인할 것, 습관 누락 문제에 대해 Soft Constraint 도입을 고려할 것, 그리고 Gemini 생성 메시지와 로컬 정적 메시지 간 톤 일관성 유지를 위한 프롬프트 엔지니어링이 필요하다는 기술적 피드백을 받았으며, 해당 사항들을 현재 순차적으로 반영 중에 있다.

기술적 고도화 계획

기술적 고도화는 네 가지 방향으로 진행된다. 첫째, **KoELECTRA** 파인튜닝을 통해 한국어 구어체 일기 텍스트에 대한 감정 분류 정확도를 개선하고, 복합 감정 인식 능력을 강화한다. 둘째, **Gemini API** 프롬프트를 최적화하여 문장 생성 비용을 절감하면서도 사용자의 변화 단계에 맞는 구체적이고 공감적인 코칭 메시지를 안정적으로 생성하며, 특히 동적 메시지와 정적 메시지 간의 어조 일관성을 확보하기 위한 프롬프트 가이드라인을 수립한다. 셋째, 적합도 표시 **Normalization** 문제를 해결하고 **Hard Constraint**와 **Soft Constraint**를 병행하여 추천 누락 없이 정확한 습관 매칭을 구현한다. 넷째, 감정 분석과 피드백 생성의 비동기 처리를 최적화하여 사용자 체감 응답 시간을 2초 이내로 유지한다.

5. 참고문헌

1. Ryan, R. M., & Deci, E. L. (2000). Self-determination theory and the facilitation of intrinsic motivation, social development, and well-being. *American Psychologist*, 55(1), 68–78. <https://psycnet.apa.org/record/2000-13324-007>
2. Nahum-Shani, I., et al. (2018). Just-in-Time Adaptive Interventions (JITAs) in Mobile Health: Key Components and Design Principles for Ongoing Health Behavior Support. *JMIR mHealth and uHealth*, 6(5), e117. <https://mhealth.jmir.org/2018/5/e117/>
3. Dijkstra, A., & De Vries, H. (2012). Personalized Persuasion: Tailoring Digital Health Interventions to User Characteristics. *Psychological Science*. <https://journals.sagepub.com/doi/10.1177/0956797611436349>
4. Grand View Research. (2024). Wellness Apps Market Size, Share & Trends Analysis Report By Type, By Platform, By Device, By Subscription, And Segment Forecasts, 2025–2030. <https://www.grandviewresearch.com/industry-analysis/wellness-apps-market-report>
5. Schwabe, L., & Wolf, O. T. (2009). Stress Prompts Habit Behavior in Humans. *Journal of Neuroscience*, 29(22), 7191–7198. <https://www.jneurosci.org/content/29/22/7191>
6. Business of Apps. (2026). App Retention Rates 2026. <https://www.businessofapps.com/data/app-retention-rates/>
7. GetStream. (2026). 2026 Guide to App Retention: Benchmarks, Stats, and More. <https://getstream.io/blog/app-retention-guide/>
8. Amra and Elma. (2025). Top Mobile App Retention Statistics 2025. <https://www.amraandelma.com/mobile-app-retention-statistics/>
9. monologg. (n.d.). KoELECTRA: Korean ELECTRA Pre-trained Language Model. *Hugging Face*. <https://huggingface.co/monologg/koelectra-base-v3-discriminator>
10. Google. (n.d.). Gemini API Documentation. <https://ai.google.dev/gemini-api/docs>
11. Recombee. (n.d.). Recombee Recommendation Engine. <https://www.recombee.com/>